



Frankfurt University of Applied Sciences

Fachbereich 2: Informatik und Ingenieurwissenschaften

Masterarbeit

Wissenschaftliche Berichtsabgabe zur Masterarbeit

Thema:

„KI-basierte Anomalieerkennung auf ressourcenbeschränkter Edge-Hardware“

Erster Prüfer: Herr Prof. Dr. Karsten Schmidt

Zweite Prüferin: Frau M.Sc. Virginia Kramer

Studiengang: Mechatronik und Robotik

Bearbeiter: Abdulmalik Alzubi

Matrikelnummer: 1224063

Zeitraum: 26.05.2026 – 27.10.2026

Abgabedatum: 27.10.2026

Erklärung

Hiermit erkläre ich, dass ich die vorliegende Arbeit selbständig erstellt und keine anderen als die angegebenen Hilfsmittel benutzt habe.

Soweit ich auf fremde Materialien, Texte oder Gedankengänge zurückgegriffen habe, enthalten meine Ausführungen vollständige und eindeutige Verweise auf die Urheber und Quellen.

Alle weiteren Inhalte der vorgelegten Arbeit stammen von mir im urheberrechtlichen Sinn, soweit keine Verweise und Zitate erfolgen.

Datum: Frankfurt, den _____

Unterschrift: _____

Abdulmalik Alzubi

Zusammenfassung

Diese Arbeit vergleicht RMS-Schwelle, Isolation Forest und TFLite-Autoencoder zur lokalen Anomalieerkennung an einem Lüfteraufbau. Die Hardware bleibt ein Raspberry Pi 5 mit ADXL345 und einem extern versorgten Lüfter bei 75 % PWM. Im aktuellen Versuch wird eine äußere Anregung durch ein auf dem Tisch vibrierendes Handy erzeugt. Alle Verfahren verarbeiten nicht überlappende Fenster mit 128 XYZ-Punkten. Modelle, Skalierung und P99-Schwellen des am 18. September 2026 erstellten Normalprofils bleiben während der Handyversuche unverändert.

Am 19. September 2026 wurden drei ausgewählte Einzelaufnahmen und ein gemeinsamer Live-Lauf ausgewertet. Im gemeinsamen Lauf erhalten die drei Verfahren dieselben 197 Rohfenster. Der Autoencoder gibt den ersten Alarm aus; Isolation Forest folgt 19,061 ms später im selben Fenster 108. RMS überschreitet seine Schwelle erst in Fenster 109 und gibt den ersten Alarm 615,659 ms nach dem Autoencoder aus. Damit zeigt der Autoencoder in diesem Lauf den günstigsten beobachteten Alarmausgabezeitpunkt. Im Replay der drei Einzelaufnahmen liegt er einmal ein Fenster vor beiden Baselines und zweimal im gleichen ersten Alarmfenster. Diese Ergebnisse beschreiben wenige Ereignisse und keine allgemeine Überlegenheit.

Der gemeinsame Prozess benötigt einen CPU-Median von 13,04 % eines logischen Kerns und maximal 174,42 MiB abgetasteten RSS. RMS hat den geringsten, Isolation Forest den höchsten Verarbeitungsaufwand. Elektrische Leistung und Energie wurden nicht gemessen. Die Aufforderung zum Anrufen und die Rückmeldung zum Ende sind keine physikalischen Zeitreferenzen; eine absolute Erkennungsverzögerung, F1 oder bestätigte Fehlalarmrate lassen sich aus der Handyreihe daher nicht bestimmen. Frühere Versuche mit einem anderen Modell- und Vorverarbeitungsstand bleiben als getrennte Prüfreihe erhalten, einschließlich ihrer negativen Erkennungsergebnisse. Der Bericht ist eine nachvollziehbare Pilotstudie mit dokumentierten Aussagegrenzen.

Inhaltsverzeichnis

Erklärung.....	i
Zusammenfassung.....	ii
Abbildungsverzeichnis.....	v
Tabellenverzeichnis.....	vi
Abkürzungsverzeichnis.....	vii
1 Einleitung, Problemstellung und Zielsetzung.....	1
1.1 Ausgangssituation und Motivation.....	1
1.2 Wissenschaftliche Problemstellung.....	1
1.3 Zielsetzung der Arbeit.....	1
1.4 Forschungsfragen und Hypothesen.....	2
1.5 Abgrenzung der Arbeit.....	3
1.6 Aufbau der Arbeit.....	3
2 Theoretische Grundlagen.....	4
2.1 Zustandsüberwachung und Anomalieerkennung.....	4
2.2 Verfahrensklassen für die Anomalieentscheidung.....	4
2.2.1 Statistische Schwellenwertverfahren.....	4
2.2.2 Klassische Verfahren im Normaldaten-Szenario.....	4
2.2.3 Autoencoder.....	5
2.3 Messdaten und Fensterbildung.....	5
2.4 Datenaufteilung und Vorverarbeitung.....	5
2.5 Schwellenwertkalibrierung.....	6
2.6 Erkennungsqualität und Aussagekraft der Evaluation.....	6
2.7 Lokale Ausführung und Zeitverhalten.....	7
2.8 Veränderte Normalbedingungen.....	7
3 Stand der Technik und verwandte Arbeiten.....	8
3.1 Datenbasis und Messbedingungen.....	8
3.2 Verfahren und Edge-Ausführung.....	8
3.3 Vergleich ausgewählter Arbeiten.....	9
3.4 Forschungslücke und Einordnung der eigenen Arbeit.....	9
4 Anforderungsanalyse.....	10
4.1 Messdaten und Vergleichsbasis.....	10
4.2 Lokale Verarbeitung und Zeitverhalten.....	10
4.3 Evaluation.....	11
4.4 Kernanforderungen und Nachweis.....	11
5 Variantenvergleich und Systementscheidung.....	14
5.1 Bewertungsrahmen und Vorgehen.....	14
5.2 Bewertung der Prüfstandsalternativen.....	15
5.3 Auswahl von Sensorik und Edge-Plattform.....	17
5.4 Datenbasis, Signalrepräsentation und Verfahren.....	19
5.5 Kriterienbasierte Bewertung der Softwarealternativen.....	21
5.6 Systementscheidung und Übergang zur Implementierung.....	23
6 Implementierung und aktueller Versuchsaufbau.....	24
6.1 Ziel und Abgrenzung des Handyversuchs.....	24

6.2 Hardware und mechanischer Übertragungsweg.....	24
6.3 PWM, Sensor und Zeitbasis.....	25
6.4 Vorverarbeitung und Modellstände.....	26
6.5 Normaltraining, Schwellen und Exportprüfung.....	27
6.6 Einzelbetrieb, Replay und gemeinsame Live-Auswertung.....	28
6.7 Ablauf, Bedienmarker und Annahmen.....	29
6.8 Dateien, Visualisierung und Nachvollziehbarkeit.....	30
7 Evaluation.....	31
7.1 Umfang und Auswahl der Aufnahmen.....	31
7.2 Datenintegrität und Reproduzierbarkeit.....	31
7.3 Einzelbetrieb und Vergleich auf identischen Replay-Fenstern.....	32
7.4 Gemeinsamer Live-Lauf: Signal und Alarmreihenfolge.....	33
7.5 Warum unterscheiden sich die ersten Alarme?.....	35
7.6 Rechenaufwand und Verhalten des Raspberry Pi.....	36
7.7 Einordnung der früheren Prüfreihe.....	39
7.8 Forschungsfragen und Nachweisstand.....	40
8 Diskussion.....	42
8.1 Beobachteter Vorteil des Autoencoders.....	42
8.2 Schwellen, Alarmhäufigkeit und faire Bewertung.....	42
8.3 Vergleichbarkeit und interne Gültigkeit.....	43
8.4 Raspberry-Pi-Bedarf und Grenzen der Übertragung.....	43
9 Ausblick.....	44
10 Fazit.....	45
Anhang.....	46
Digitale Belege und Fassungszuordnung.....	46
Erhaltene historische Erkennungsergebnisse.....	47
Erhaltene historische Raspberry-Pi-Messwerte.....	48
Lesen der aktuellen Messdateien.....	49
KI-Unterstützung und fachliche Verantwortung.....	50
Literaturverzeichnis.....	51

Abbildungsverzeichnis

Abbildung 6-1: Schematischer Aufbau des Handyversuchs. Die Darstellung erklärt den vorgesehenen Übertragungsweg; sie ist kein Foto und bestätigt keine nicht vermessene Geometrie. Quelle: eigene Darstellung anhand von B12.....	25
Abbildung 6-2: Datenfluss des gemeinsamen Live-Vergleichs. Alle drei Methoden erhalten dasselbe Rohfenster; ihre Verarbeitung und Ausgabezeitpunkte bleiben getrennt protokolliert. Quelle: eigene Darstellung anhand der Implementierung und Journale in B12.....	28
Abbildung 7-1: Erstes Alarmfenster im Offline-Replay. Die Zahlen an den Punkten sind Fensterindizes; die vertikale Achse zeigt den Abstand zum frühesten Alarmfenster derselben Aufnahme. Eine Null bedeutet Gleichstand auf Fensterebene, keine gleichzeitig gemessene Live-Ausgabe. Quelle: B12, replay_v1/summary.json.....	33
Abbildung 7-2: Gemeinsamer Rohdaten- und Scoreverlauf. Graue Bereiche liegen zwischen den Bedienmarkern, nicht zwischen unabhängig gemessenem Vibrationsbeginn und -ende. Die Scoreachse ist logarithmisch und auf die jeweilige Schwelle normiert; der Wert 1 kennzeichnet die Alarmgrenze. Quelle: B12, Gemeinsam01.....	34
Abbildung 7-3: Nummerierte Reihenfolge der ersten Alarmausgaben im gemeinsamen Lauf und zugehörige Fensterindizes. Null ist der Zeitpunkt des ersten Autoencoder-Alarms, nicht der physische Störungsbeginn. AE und IF sind auf Fensterebene gleichauf; die Ausgabezeitpunkte unterscheiden sich. Quelle: B12, Entscheidungen von Gemeinsam01.....	35
Abbildung 7-4: Schwellenübergang der Fenster 107 bis 109. Die Normierung erleichtert das Ablesen der jeweiligen Grenzüberschreitung; die Quotienten sind weder Wahrscheinlichkeiten noch untereinander kalibrierte Störungsschweregrade. Quelle: B12, Gemeinsam01.....	36
Abbildung 7-5: Verarbeitungszeiten im Einzelbetrieb und im gemeinsamen Prozess. Symbole zeigen Median, rechte Striche P95; die Zeitachse ist logarithmisch. Erfasst wird der instrumentierte Auswerteraufwurf einschließlich seiner Vorverarbeitung, nicht eine einheitlich isolierte Modellkernzeit. Quelle: B12, ausgewählte decisions.csv.....	37
Abbildung 7-6: CPU und RAM im Einzelbetrieb und gemeinsamen Live-Prozess. Es handelt sich um Prozessmessungen mit unterschiedlichen Laufdauern; die gemeinsame Auslastung darf nicht pro Methode aufgeteilt oder aus den Einzelwerten addiert werden. Quelle: B12, decisions.csv und resources.csv.....	38
Abbildung 7-7: Zeitlicher Ressourcenverlauf des gemeinsamen Prozesses. Grau: Zeitraum zwischen Bedienmarkern. Temperatur und CPU-Takt sind zusätzliche Betriebsdiagnosen, keine Leistungs- oder Energiemessung. Quelle: B12, Gemeinsam01/resources.csv.....	39

Tabellenverzeichnis

Tabelle 2-1: Kennzahlen der binären Anomalieerkennung.....	6
Tabelle 3-1: Ausgewählte Vorarbeiten und ihre Bedeutung für die Untersuchung.....	9
Tabelle 4-1: Anforderungen, Prioritäten und Nachweise.....	12
Tabelle 5-1: Bewertung der Prüfstandsalternativen.....	15
Tabelle 5-2: Kriterien und Gewichte für den Prüfstand.....	16
Tabelle 5-3: Gewichtete Bewertung der Prüfstandsalternativen.....	16
Tabelle 5-4: Vergleich der Sensorik und Messketten.....	17
Tabelle 5-5: Bewertung der Edge-Plattformen.....	18
Tabelle 5-6: Kriterien und Gewichte für die Edge-Plattform.....	18
Tabelle 5-7: Gewichtete Bewertung der Edge-Plattformen.....	19
Tabelle 5-8: Bewertung der Signalrepräsentationen.....	19
Tabelle 5-9: Auswahl der Anomalieverfahren.....	20
Tabelle 5-10: Kriterienbasierter Vergleich der Softwarealternativen.....	21
Tabelle 5-11: Kriterien und Gewichte für die Softwareauswahl.....	22
Tabelle 5-12: Gewichtete Bewertung der Softwarealternativen.....	22
Tabelle 5-13: Systementscheidungen und ausstehende Nachweise.....	23
Tabelle 6-1: Hardware und Betriebszustand der Handyreihe.....	25
Tabelle 6-2: Festgelegte Skalierung des aktuellen Normalprofils, gerundet.....	26
Tabelle 6-3: Architektur des verwendeten Autoencoders.....	27
Tabelle 6-4: Eingefrorene Schwellen der Handyreihe.....	27
Tabelle 6-5: Zeitbegriffe und zulässige Interpretation.....	29
Tabelle 6-6: Annahmen und offene Nachweise.....	30
Tabelle 7-1: Erhaltene Anläufe und Auswertungsrolle.....	31
Tabelle 7-2: Ausgewählte Aufnahmen und Verteilung der gültigen Fenster.....	32
Tabelle 7-3: Replay derselben Rohfenster; erstes Alarmfenster und gesamte Alarmfenster.....	32
Tabelle 7-4: Erste Alarmer im gemeinsamen Live-Lauf.....	34
Tabelle 7-5: Unmittelbarer Schwellenvergleich auf dem gemeinsamen Fenster 108.....	35
Tabelle 7-6: Verarbeitungszeit im jeweiligen Auswerteraufwurf; ms.....	37
Tabelle 7-7: Prozessressourcen der ausgewählten Handyaufnahmen.....	38
Tabelle 7-8: Bewertung der Forschungsfragen und Hypothesen.....	40
Tabelle 7-9: Rückbindung an die unveränderten Anforderungen aus Kapitel 4.....	41
Tabelle A-1: Nachweisverzeichnis.....	46
Tabelle A-2: Historische Verwechslungszahlen der einen Plattenfolge auf identischen Fenstern.....	47
Tabelle A-3: Historische deskriptive Kennzahlen für das Zustandslabel; Prozent.....	47
Tabelle A-4: Historische Zählung je Aufnahme und Methode im Abschnitt [180,300) s.....	48
Tabelle A-5: Historische Latenzen; Bereich von drei getrennten Prozessen je Zeile.....	48
Tabelle A-6: Historische Ressourcen im aktiven Abschnitt [180,300) s; Bereich von drei Prozessen.....	49

Abkürzungsverzeichnis

Abkürzung	Bedeutung
AE	Autoencoder
CPU	Central Processing Unit
DFT	Diskrete Fourier-Transformation
FFT	Fast Fourier Transform
FPR	False-Positive-Rate
GPIO	General Purpose Input/Output
GUI	Graphical User Interface (grafische Benutzeroberfläche)
I ² C	Inter-Integrated Circuit
IEPE	Integrated Electronics Piezo-Electric
KI	Künstliche Intelligenz
MCU	Microcontroller Unit
ML	Machine Learning
MSE	Mean Squared Error
ODR	Output Data Rate
PWM	Pulsweitenmodulation
RAM	Random Access Memory
RMS	Root Mean Square
RSS	Resident Set Size
SBC	Single-Board Computer
SPI	Serial Peripheral Interface
TFLite	TensorFlow Lite
AC	Wechselanteil nach Entfernung des Gleichanteils
IF	Isolation Forest
LSB	Least Significant Bit
MiB	Mebibyte (2 ²⁰ Byte)
P99	Empirisches 99. Perzentil
PSD	Power Spectral Density (Leistungsdichtespektrum)

1 Einleitung, Problemstellung und Zielsetzung

1.1 Ausgangssituation und Motivation

Beschleunigungssignale ermöglichen es, Schwingungen technischer Systeme zu beobachten. Bei rotierenden Komponenten können Veränderungen dieser Signale auf einen veränderten Zustand hinweisen. Randall und Antoni (2011) erläutern dies am Beispiel der Wälzlagerdiagnose. Die vorliegende Arbeit untersucht die lokale Erkennung solcher Abweichungen anhand von Beschleunigungsdaten.

Die Auswertung soll lokal in der Nähe des Sensors erfolgen. Dieses Prinzip des Edge Computing kann die Übertragung von Rohdaten und die Abhängigkeit von entfernten Rechendiensten verringern (Shi et al., 2016). Neben einer guten Erkennung sind deshalb der Rechen- und Speicherbedarf sowie die Verarbeitungszeit wichtig. Zusätzlich wird untersucht, wie sich veränderte normale Messbedingungen auf die Entscheidungen auswirken.

1.2 Wissenschaftliche Problemstellung

Bei der normalitätsbasierten Anomalieerkennung wird ein Referenzmodell aus normalen Messdaten gebildet. Neue Daten gelten als auffällig, wenn sie ausreichend von dieser Referenz abweichen. Dieses Vorgehen benötigt keine vollständige Sammlung möglicher Fehlerbeispiele. Seine Aussagekraft hängt jedoch davon ab, welche Normalbedingungen die Referenzdaten abdecken (Chandola et al., 2009).

Für einen belastbaren Vergleich müssen die Verfahren unter vergleichbaren Bedingungen bewertet werden. Informationen aus dem Test dürfen weder die Modellwahl noch die Vorverarbeitung oder die Entscheidungsschwelle beeinflussen; andernfalls kann die Erkennungsleistung überschätzt werden (Kaufman et al., 2012). Außerdem muss die gesamte Verarbeitung auf der Zielhardware betrachtet werden. Eine kurze Modellinferenz allein zeigt noch nicht, ob alle eingehenden Daten rechtzeitig verarbeitet werden.

1.3 Zielsetzung der Arbeit

Ziel ist die Konzeption, Umsetzung und experimentelle Bewertung einer lokalen Verarbeitungskette (Pipeline) für mehrkanalige Beschleunigungsdaten. Verglichen werden ein neuronales Hauptverfahren, eine statistische Schwellenwertbaseline und ein klassisches Lernverfahren. Auch das klassische Verfahren gehört zum maschinellen Lernen. Die konkrete Auswahl des Systems und des Versuchsaufbaus wird in Kapitel 5 anhand der Anforderungen und eines Variantenvergleichs begründet.

Der Beitrag ist ein Methodenvergleich auf identischen Sensoreingaben mit nachvollziehbarer Schwellenkalibrierung und Messung des technischen Aufwands. Der aktuelle Schwerpunkt liegt auf der Reihenfolge der Alarmausgaben bei einer Handyvibration. Frühere Prüfungen zur Erkennungsqualität und zu geänderten Normalbedingungen werden getrennt eingeordnet; sie werden durch den neuen Versuch nicht nachträglich umgedeutet.

1.4 Forschungsfragen und Hypothesen

Zentrale Forschungsfrage: Wie lässt sich eine lokale Anomalieerkennung für mehrkanalige Beschleunigungsdaten auf ressourcenbeschränkter Edge-Hardware umsetzen und hinsichtlich Erkennungsqualität, Ressourcenbedarf, Zeitverhalten und veränderter Normalbedingungen bewerten?

FF1: Welche Anforderungen ergeben sich aus der Aufgabe, und wie begründen sie die Auswahl von Versuchsaufbau, Sensorik, Hardware und Verfahren?

FF2: Wie unterscheiden sich Hauptverfahren und Baselines bei gleichen Testfenstern und einer einheitlichen Schwellenwertkalibrierung auf Normaldaten hinsichtlich Precision, Recall, F1-Score und Fehlalarmrate? Ergänzend wird in der Handyreihe deskriptiv untersucht, welche Methode zuerst ein Alarmfenster markiert und ihre Alarmausgabe bereitstellt. Diese ergänzende Zeitfrage ersetzt die ursprüngliche Qualitätsfrage nicht.

FF3: Wie viel Speicher und Rechenleistung benötigt die lokale Pipeline, und wie häufig überschreitet ihre Verarbeitung das Zeitbudget bis zum nächsten Fenster?

FF4: Wie verändert sich die Fehlalarmrate bei gezielt geänderten Normalbedingungen, wenn Vorverarbeitung, Modell und Schwelle unverändert bleiben?

Die ursprünglichen Arbeitshypothesen bleiben als Bewertungsmaßstab erhalten. Der Bericht unterscheidet die frühere Prüfreihe mit definierten Zustandslabels und die nachfolgende explorative Handyreihe mit Bedienmarkern. Training und Schwellenkalibrierung der Handyreihe erfolgen auf vorher aufgenommenen Normaldaten; an den Störungsaufnahmen werden keine Schwellen angepasst. Wegen fehlender unabhängiger Zeitreferenz und weniger Ereignisse liefert die neue Reihe keinen zusätzlichen vollständigen Test von H1 oder H3. Die ergänzende Rangfolge der Alarmausgaben ist ein deskriptiver Befund, keine rückwirkend vorab formulierte Hypothese.

H1 – Erkennungsqualität: Das neuronale Hauptverfahren erreicht bei den definierten Versuchsanomalien einen höheren F1-Score als die statistische Baseline. Diese Erwartung beruht auf der möglichen Nutzung von Informationen, die ein einzelner Kennwert nicht abbildet. Bewertet werden die F1-Differenz auf derselben Testmenge sowie Fehlalarme und übersehene Anomalien.

H2 – Zeitverhalten: Das empirische 99. Perzentil (P_{99}) der Verarbeitungslatenz liegt unter dem Fensterintervall S/f_s . Die Latenz reicht vom vollständigen Fenster bis zur verfügbaren Entscheidung. S bezeichnet die Schrittweite in Messzeitpunkten, f_s die aus Zeitstempeln ermittelte Erfassungsrate. Fristüberschreitungen werden zusätzlich gezählt; P_{99} garantiert keine fristgerechte Ausgabe jeder Entscheidung.

H3 – Veränderte Normalbedingungen: Bei unveränderter Pipeline steigt die Fehlalarmrate unter der untersuchten veränderten Normalbedingung. Verglichen werden die Fehlalarmraten vor und nach der Änderung. Eine Ursache lässt sich nur bei kontrollierten weiteren Einflüssen zuordnen.

1.5 Abgrenzung der Arbeit

Die Ergebnisse gelten für die untersuchten Bedingungen und die im Versuchsprotokoll definierten Anomalien. Deren Erkennung erlaubt weder eine gesicherte Schadensdiagnose noch eine Prognose der Restlebensdauer. Die Übertragung auf andere Systeme und den industriellen Dauerbetrieb erfordert weitere Untersuchungen.

Der Kernversuch bewertet die lokale Inferenz eines zuvor trainierten Modells auf einer Zielplattform. Die Pipeline bleibt dabei unverändert. Eine mögliche spätere Nachkalibrierung wäre als eigener Versuch auszuwerten. Sie ist nicht Bestandteil des hier abgeschlossenen eingefrorenen Vergleichs.

Die überarbeitete Fassung berücksichtigt die Handyversuche vom 19. September 2026 und das zugehörige Normalprofil vom 18. September. Die Hardwareauswahl bleibt unverändert. Frühere Messungen vom 11./12. September mit separater Platte und einem anderen Modellstand werden als historische Prüfreihe kenntlich gemacht und im Anhang nachgewiesen. Die Handyreihe verwendet keine Platte. Nicht vorhandene Wiederholungen, Drehzahlmessungen, physische Vibrationszeitreferenzen und elektrische Verbrauchsmessungen werden nicht als erbracht behandelt. Teilnachweise der Messqualität und Wiederholbarkeit werden nicht nachträglich zu vollständiger Erfüllung erklärt.

1.6 Aufbau der Arbeit

Kapitel 2 und 3 erläutern Grundlagen und Vorarbeiten. Kapitel 4 und 5 begründen Anforderungen und Hardwareauswahl. Kapitel 6 beschreibt den aktuellen Handyaufbau, die Implementierung und das Protokoll. Kapitel 7 trennt Einzelbetrieb, Replay und gemeinsamen Live-Vergleich einschließlich Raspberry-Pi-Ressourcen. Diskussion, Ausblick und Fazit ordnen die Ergebnisse ein. Der Anhang erhält die früheren Prüfergebnisse und verknüpft alle digitalen Nachweise.

2 Theoretische Grundlagen

Dieses Kapitel erklärt die benötigten Begriffe, mögliche Verfahren und die Messgrößen für die Evaluation. Die dargestellten Ansätze bilden die Grundlage für den Variantenvergleich; ihre Auswahl für das System erfolgt erst in Kapitel 5.

2.1 Zustandsüberwachung und Anomalieerkennung

Zustandsüberwachung (Condition Monitoring) erfasst und bewertet den technischen Zustand. Diagnose ordnet Fehlerursachen zu, Prognose schätzt die künftige Zustandsentwicklung (Jardine et al., 2006). Anomalieerkennung bewertet zunächst nur Abweichungen von einer Normalreferenz.

Was als anomal gilt, hängt von Referenz und Betriebskontext ab. Auch ein zeitlicher Verlauf kann auffällig sein, obwohl seine Einzelwerte unauffällig sind (Chandola et al., 2009). Deshalb werden hier Messfenster bewertet.

Klassische und neuronale Verfahren können auf als normal ausgewählten Daten angepasst werden. Dieses Normaldaten-Szenario beschreibt die Trainingsgrundlage und ist von der Algorithmusklasse zu unterscheiden. Der Anomalie-Score ist eine Bewertungsgröße, zunächst keine Schadenswahrscheinlichkeit (Ruff et al., 2021).

2.2 Verfahrensklassen für die Anomalieentscheidung

2.2.1 Statistische Schwellenwertverfahren

Der Effektivwert (Root Mean Square, RMS) eines Kanals mit N Messwerten ist die Quadratwurzel des Mittelwerts der quadrierten Werte:

$$RMS(x) = \sqrt{\frac{1}{N} \sum_{n=0}^{N-1} x[n]^2} \quad (2-1)$$

Der RMS fasst die Amplitude eines Fensters zusammen, erhält aber die Reihenfolge der Werte nicht. Für den Fenstermittelwert $\bar{x}$ gilt $RMS^2(x) = \bar{x}^2 + RMS^2(x - \bar{x})$. Ohne Mittelwertentfernung beeinflussen daher auch konstante Anteile den Kennwert. Beschleunigungssensoren erfassen neben Bewegung unter anderem die Gravitation und einen Sensoroffset (Analog Devices, 2022). Rohwert-RMS und RMS nach Mittelwertentfernung sind deshalb zu unterscheiden. Ebenso muss feststehen, wie mehrere Messkanäle zusammengefasst werden.

2.2.2 Klassische Verfahren im Normaldaten-Szenario

Isolation Forest nutzt zufällige Teilungsbäume. Jeder Schritt teilt die Daten anhand eines zufällig gewählten Merkmals und Teilungspunkts. Beobachtungen, die im Mittel durch wenige Teilungen isoliert werden, gelten als auffälliger (Liu et al., 2008). Im Normaldaten-Szenario erfolgt seine Anpassung ausschließlich anhand normaler Beobachtungen.

Die Eingabe ist ein Vektor fester Länge aus Fensterkennwerten oder aneinandergereihten Messwerten. Die verwendete Darstellung und die Richtung des ausgegebenen Scores müssen dokumentiert werden.

2.2.3 Autoencoder

Ein Autoencoder erzeugt aus einer Eingabe x eine Rekonstruktion $\hat{x}$. Der Encoder bildet die Eingabe auf eine interne Darstellung ab, der Decoder rekonstruiert daraus die Ausgangswerte. Eine begrenzte Modellkapazität oder Regularisierung soll bloßes Kopieren verhindern (Goodfellow et al., 2016). Als Anomalie-Score kann der mittlere quadratische Rekonstruktionsfehler verwendet werden:

$$s(x) = MSE(x, \hat{x}) = \frac{1}{d} \sum_{j=1}^d (x_j - \hat{x}_j)^2 \quad (2-2)$$

Dabei bezeichnet d die Zahl der verglichenen Einzelwerte. Für ein Fenster mit H Messzeitpunkten und C Kanälen gilt $d = HC$. Der Score hängt von der Skalierung und der Mittelung ab. Bei standardisierten Eingaben ist er dimensionslos.

Die Annahme lautet, dass ein auf Normaldaten trainierter Autoencoder auffällige Eingaben schlechter rekonstruiert. Das ist keine allgemeine Garantie: Auch bestimmte Anomalien können gut rekonstruiert werden (Ruff et al., 2021). Ob der Score die definierten Versuchsanomalien trennt, muss daher experimentell geprüft werden.

2.3 Messdaten und Fensterbildung

Bei dreiachsigen Messungen enthält jeder Messvektor beispielsweise einen X-, Y- und Z-Wert. Sensorausgaberate, Signalbandbreite und Abfragerate der Software sind unterschiedliche Größen. Häufigeres Auslesen erzeugt nicht automatisch neue Sensorwerte. Die Abstimmung von Sensorkonfiguration und Datenabruf ist daher zu prüfen (Analog Devices, 2022).

Ein Fenster bündelt H aufeinanderfolgende Messvektoren. Bei C Messkanälen enthält es HC Werte. Die Schrittweite S bestimmt, nach wie vielen neuen Messzeitpunkten das nächste Fenster beginnt. Bei gleichmäßiger Erfassung beträgt das Fensterintervall S/f_s . Für die Zeitbewertung wird f_s aus den Zeitstempeln bestimmt; ob jeder erfasste Vektor eine neue Sensormessung enthält, wird getrennt geprüft.

Bei ungleichmäßiger Erfassung und Datenlücken reichen nominelle Zeitangaben nicht aus. Die tatsächlich erfassten Zeitstempel müssen deshalb erhalten bleiben.

2.4 Datenaufteilung und Vorverarbeitung

Trainingsdaten dienen zur Anpassung des Modells. Validierungsdaten unterstützen die Auswahl von Einstellungen und die Kalibrierung. Die Testdaten bleiben bis zur abschließenden Bewertung ungenutzt. Vollständige Aufnahmen bilden dabei Gruppen und

werden vor der Fensterbildung getrennt zugeordnet. So werden ähnliche Abschnitte derselben Aufnahme nicht auf Training und Test verteilt (scikit-learn developers, o. J. b).

Data Leakage liegt vor, wenn Informationen in die Entwicklung einfließen, die bei der vorgesehenen Anwendung nicht verfügbar wären. Ein Beispiel ist die Standardisierung mit Mittelwert und Standardabweichung der gesamten Datenmenge. Solche Parameter dürfen nur aus Trainingsdaten bestimmt werden. Danach werden sie unverändert auf Validierungs-, Test- und Live-Daten angewendet (scikit-learn developers, o. J. a).

Fenster derselben Aufnahme sind keine unabhängigen Messläufe. Auch getrennte Aufnahmen können bei gleicher Montage sehr ähnlich sein. Die Aussagekraft hängt daher von Zahl und Vielfalt der Messläufe ab.

2.5 Schwellenwertkalibrierung

Aus dem Anomalie-Score wird durch einen Schwellenwert eine Entscheidung. Für den Vergleich werden die Scores so ausgerichtet, dass größere Werte eine stärkere Auffälligkeit bedeuten. Die Entscheidung lautet dann anomal, wenn der Score die festgelegte Schwelle überschreitet.

Eine mögliche Kalibrierung nutzt ein hohes empirisches Quantil der Scores separater Normaldaten. Ein Quantil bezeichnet einen Grenzwert in der sortierten Verteilung. Die konkrete Quantilsregel wird für alle Verfahren vor der Testauswertung festgelegt. Damit wird dieselbe Kalibrierungsregel verwendet, auch wenn die numerischen Schwellen verschieden sind.

Eine so bestimmte Schwelle garantiert keine bestimmte Fehlalarmrate auf späteren Aufnahmen. Dafür müssen die neuen Normaldaten ausreichend zur Kalibrierungsgrundlage passen. Eine nachträgliche Anpassung anhand der Testgüte würde die unabhängige Bewertung aufheben (scikit-learn developers, o. J. a).

2.6 Erkennungsqualität und Aussagekraft der Evaluation

Die positive Klasse ist die definierte Anomalie. Die Konfusionsmatrix zählt richtig erkannte Anomalien (TP), Fehlalarme (FP), übersehene Anomalien (FN) und richtig erkannte Normalfenster (TN). Daraus ergeben sich die Kennzahlen in Tabelle 2-1 (scikit-learn developers, o. J. c).

Tabelle 2-1: Kennzahlen der binären Anomalieerkennung

Kennzahl	Berechnung	Bedeutung
Precision	$TP / (TP + FP)$	Anteil richtiger Anomalien unter allen Alarmen
Recall	$TP / (TP + FN)$	Anteil erkannter Anomalien unter allen tatsächlichen Anomalien
F1-Score	$2TP / (2TP + FP + FN)$	Harmonisches Mittel von Precision und Recall

Kennzahl	Berechnung	Bedeutung
Fehlalarmrate (FPR)	$FP / (FP + TN)$	Anteil falsch gemeldeter Fenster unter allen Normalfenstern
Accuracy	$(TP + TN) / (TP + TN + FP + FN)$	Anteil aller richtigen Entscheidungen

Eine hohe Accuracy kann bei ungleichen Klassenanteilen täuschen. Deshalb werden die Konfusionsmatrix und mehrere Kennzahlen gemeinsam berichtet. Nicht definierte Werte, etwa Recall ohne Anomalien im Datensatz, werden gekennzeichnet. Für reine Normalaufnahmen steht die Fehlalarmrate im Mittelpunkt. Zusätzlich werden Klassenanteile und Ergebnisse je Aufnahme angegeben.

2.7 Lokale Ausführung und Zeitverhalten

Bei lokaler Ausführung wird das trainierte Modell nahe der Datenquelle ausgewertet (Shi et al., 2016). Ob Rechenleistung und Arbeitsspeicher der Zielplattform ausreichen, muss für die konkrete Anwendung gemessen werden.

Die Modellgröße bezeichnet die gespeicherte Datei. Der Prozessspeicher umfasst zusätzlich unter anderem Bibliotheken, Eingabedaten und Zwischenergebnisse. Modellgröße und Speicherbedarf während der Ausführung sind daher getrennt auszuweisen. Auch die CPU-Auslastung benötigt eine dokumentierte Messdauer und Bezugsgröße.

Die Inferenzzeit umfasst nur die Modellauswertung. Die Verarbeitungslatenz reicht in dieser Arbeit vom vollständigen Fenster bis zur verfügbaren Entscheidung und enthält auch Vorverarbeitung und Wartezeiten. Die Zeit zum Sammeln des Fensters ist darin nicht enthalten. Ein Echtzeitsystem muss seine Ergebnisse innerhalb festgelegter Fristen liefern (Buttazzo, 2011). Mittelwerte allein belegen das nicht.

P95 und P99 beschreiben hohe empirische Perzentile der gemessenen Latenzen. Sie werden zusammen mit Fristüberschreitungen und Erfassungspausen ausgewertet. Davon zu unterscheiden sind das erste als auffällig markierte Fenster und der Zeitabstand zwischen den ersten Alarmausgaben mehrerer Methoden. Eine Verzögerung ab physischem Störungsbeginn setzt eine unabhängige Zeitreferenz voraus. Trainierte Modelle können für die lokale Ausführung in ein anderes Format konvertiert werden (Google, 2026a). Danach müssen Scores und Entscheidungen erneut geprüft werden.

2.8 Veränderte Normalbedingungen

Ändern sich Montage, Sensororientierung oder Betriebspunkt, können sich normale Eingabedaten von der bisherigen Referenz unterscheiden. Solche Verteilungsänderungen werden häufig als Domain Shift bezeichnet. Sie können die Anomalieerkennung beeinträchtigen (Ruff et al., 2021).

FF4 untersucht deshalb die Fehlalarmrate bei unveränderter Pipeline unter geänderten Normalbedingungen. Eine anschließende Nachkalibrierung ist als eigener Versuch auszuwerten.

3 Stand der Technik und verwandte Arbeiten

Dieses Kapitel vergleicht ausgewählte Originalstudien nach Datenbasis, Lernaufgabe, lokaler Ausführung und Übertragbarkeit. Die gezielte Auswahl mit Recherchestand September 2026 umfasst Arbeiten zu den Forschungsfragen; sie ist keine vollständige systematische Literaturübersicht.

3.1 Datenbasis und Messbedingungen

Lessmeier et al. (2016) untersuchen die überwachte Unterscheidung gesunder Lager sowie von Innen- und Außenringschäden. Sie vergleichen künstlich erzeugte Schäden mit Schäden aus beschleunigten Lebensdauerversuchen. Die Klassifikationsgüte hängt dabei von Schadensherkunft und Verfahren ab. Der Befund betrifft überwachte Lagerklassifikation und lässt sich nicht unmittelbar auf normaldatenbasiertes Training übertragen.

Pittino et al. (2020) untersuchen eine Waferbearbeitungsmaschine bei unterschiedlichen Drehzahlen und Verschleißzuständen. Normaldaten stammen aus funktionsfähigen Kammern, Fehlerdaten aus einem Demonstrator ohne Schmierung. Verglichen werden regressionsbasierte Regelkarten und eine überwachte Support-Vector-Machine-Klassifikation. Letztere nutzt zusätzlich Fehlerbeispiele. Ihr Ergebnis belegt deshalb keinen allgemeinen Vorteil im Normaldaten-Szenario.

Knap et al. (2026) vergleichen klassische und tiefe Klassifikationsverfahren in sechs Übertragungsszenarien auf CWRU- und Paderborner Lagerdaten. Ihr beschriebenes Protokoll trennt vollständige Aufnahmen vor der Fensterbildung. Für die eigene Untersuchung folgt daraus die Trennung ganzer Messläufe. Geänderte Normalbedingungen werden zusätzlich in einem eigenen Versuch betrachtet.

3.2 Verfahren und Edge-Ausführung

Ren et al. (2021) führen mit TinyOL einen Autoencoder auf einem Cortex-M4-Mikrocontroller aus und ermöglichen das lokale Nachlernen seiner letzten Schicht. Dreiaxige Lüftermessungen werden vor der Modelleingabe durch Hauptkomponentenanalyse auf eine Dimension reduziert. Nach erneuter Anbringung des Sensors verändert sich der Rekonstruktionsfehler; die feste Schwelle verursacht mehr Fehlalarme. Die untersuchte Modellanpassung ist von der Bewertung einer unveränderten Pipeline zu unterscheiden.

Antonini et al. (2023) entwickeln einen ESP32-Sensorknoten für die Überwachung von Unterwasserpumpen. Die Evaluation erfolgt an einer Platine auf einem gefederten PC-Lüfterprüfstand und beschreibt damit keinen gemessenen Pumpenbetrieb. Merkmalsbildung, Isolation-Forest-Training und Inferenz laufen lokal; das Training setzt normalen Betrieb voraus. Die Ressourcenmessungen unterscheiden Laden, Training und Inferenz. Die Erkennungsgenauigkeit wird nicht bewertet.

Garay et al. (2026) vergleichen vier Algorithmen in fünf Varianten, darunter Isolation Forest und einen Autoencoder, auf einem Cortex-M4F. Nach einer Sensorneumontage treten mehr Fehlalarme auf; gleichzeitig verändert sich die Umgebungstemperatur. Für die Einordnung in dieser Arbeit wird die Fehlalarmänderung deshalb nicht als isolierter Montageeffekt behandelt. Die Autoren untersuchen außerdem eine Nachkalibrierung von Skalierung und Schwelle. Beim Isolation Forest stammen die Erkennungskennzahlen vom Modell mit 100 Bäumen, die Mikrocontrollerwerte von einer Variante mit sechs Bäumen. Beide Größen beschreiben somit unterschiedliche Modellstände.

3.3 Vergleich ausgewählter Arbeiten

Tabelle 3-1 fasst die drei Studien mit eigener Hardwareausführung zusammen. Wegen unterschiedlicher Daten und Messgrenzen erlauben ihre Kennzahlen keine unmittelbare Rangfolge.

Tabelle 3-1: Ausgewählte Vorarbeiten und ihre Bedeutung für die Untersuchung

Arbeit	Daten und Lernaufgabe	Ausführung / Messung	Einordnung für diese Arbeit
Ren et al. (2021)	Lüfter; Autoencoder und Nachlernen	Cortex-M4; Inferenz und Online-Anpassung	Belegt Montageempfindlichkeit; Anpassung getrennt von fester Normalreferenz bewerten
Antonini et al. (2023)	Unterwasserpumpe als Zielanwendung; Evaluation am gefederten Lüfterprüfstand; Isolation Forest mit Normaltraining	ESP32; Laden, Training und Inferenz	Ressourcen getrennt erfassen; keine Erkennungsbewertung in der Studie
Garay et al. (2026)	Unwuchtversuch; fünf normaltrainierte Modellvarianten	Cortex-M4F; Montage- und Temperaturänderung	Modellstände abgleichen; Nachkalibrierung getrennt bewerten

3.4 Forschungslücke und Einordnung der eigenen Arbeit

Die Vorarbeiten zeigen, dass lokale Anomalieerkennung und die Anpassung an veränderte Messbedingungen bereits untersucht werden. Für die eigene Untersuchung bleibt zu klären, welchen zusätzlichen Erkennungsnutzen ein neuronales Verfahren gegenüber den beiden Baselines bietet und welchen technischen Aufwand es verursacht. Die Arbeit liefert hierzu eine anwendungsbezogene Vergleichsevaluation.

Dafür werden gleiche Testfenster und eine gemeinsame Kalibrierungsregel für FF2 sowie Messungen am tatsächlich ausgeführten Modell für FF3 verbunden. Die frühere Untersuchung geänderter Normalbedingungen adressiert FF4 separat. Die ergänzende Handyreihe vergleicht zuerst Einzelprozesse und anschließend dieselben Rohfenster im Replay und im gemeinsamen Live-Prozess. So wird zwischen unterschiedlichen physikalischen Anregungen, identischen Eingaben und zusätzlicher Konkurrenz um Rechenzeit unterschieden.

4 Anforderungsanalyse

Dieses Kapitel leitet aus den Forschungsfragen, den Grundlagen und den Grenzen der Vorarbeiten überprüfbare Anforderungen ab. Sie beschreiben die benötigten Funktionen und Bewertungskriterien unabhängig von einer konkreten technischen Lösung.

4.1 Messdaten und Vergleichsbasis

Für die Untersuchung muss ein reproduzierbarer Testaufbau realisiert werden (A11). Er muss Messungen im Normalzustand und unter gezielt hergestellten Anomaliebedingungen ermöglichen. Eine Normalbedingung muss für FF4 unabhängig von der Anomaliedefinition veränderbar sein (A10). Ein sicherer Betrieb ist Voraussetzung für die Messläufe. Versuchsbedingungen, Rücksetzbarkeit und Ablauf sind so zu dokumentieren, dass Messläufe wiederholt werden können.

Die Messdaten müssen die benötigten Beschleunigungskanäle mit Einheit, Zeitstempeln und Aufnahmekennung enthalten (A1). Messbereich, nutzbare Bandbreite, Signal-Rausch-Abstand und zeitliche Erfassung sind für die Versuchszustände zu prüfen. Akzeptanzgrenzen werden vor der abschließenden Testserie festgelegt. Datenlücken, ungültige Werte und Sättigung sind zu kennzeichnen. Für Normal- und Anomaliedaten muss feststehen, ob die Kennzeichnung ganze Aufnahmen oder einzelne Zeitabschnitte beschreibt.

Vollständige Aufnahmen werden vor der Fensterbildung getrennt Training, Validierung oder Test zugeordnet. Fenster dürfen keine Aufnahmegrenzen überschreiten. Training und Schwellenwertkalibrierung verwenden nur Normaldaten. Die Vorverarbeitung wird nach den Regeln aus Abschnitt 2.4 angepasst und unverändert auf spätere Daten angewendet.

Das neuronale Hauptverfahren und die beiden Baselines müssen dieselben Testfenster und Labels verwenden. Fensterlänge und Schrittweite werden gemeinsam festgelegt. Unterschiedliche Eingabedarstellungen und der Umgang mit unvollständigen Fenstern sind zu dokumentieren. Die Schwellen folgen derselben Kalibrierungsregel auf separaten Normaldaten. Modellwahl und Kalibrierung müssen vor der Testauswertung abgeschlossen sein.

4.2 Lokale Verarbeitung und Zeitverhalten

Datenerfassung, Vorverarbeitung und Entscheidung müssen nach dem Start automatisch und ohne Cloud-Dienst auf der Zielhardware laufen (A4). Die Softwareumgebung muss die benötigten Modelloperationen und Schnittstellen unterstützen; Abhängigkeiten und Versionen sind zu dokumentieren (A5, A6). Für jedes gültige Fenster sind Score, Schwelle und die Entscheidung normal/anomal zu speichern. Ungültige Fenster dürfen nicht als normal bewertet werden. Der Betrieb muss ohne Speicherabbruch und ohne stetig wachsenden Verarbeitungsrückstand möglich sein (A8).

Messläufe müssen gezielt gestartet und geordnet beendet werden können (A13). Jeder Lauf erhält eine eindeutige Kennung. Beim Beenden müssen die bis dahin vollständig erfassten

Daten und Ergebnisse gespeichert bleiben. Messdaten, Entscheidungen und die zugehörigen Angaben müssen maschinenlesbar vorliegen und eindeutig zusammengehören (A5).

Als optionale Erweiterung kann eine grafische Benutzeroberfläche zur Live-Anzeige (Live-GUI) entwickelt werden (A12). Sie zeigt den aktuellen Erfassungsstatus sowie Score, Schwelle, Entscheidung und Zeitstempel des zuletzt ausgewerteten Fensters. Fehlende oder veraltete Daten müssen erkennbar sein. Die Anzeige verwendet die Ergebnisse der Verarbeitung und verändert weder Modell noch Schwelle. Der Messbetrieb und die Evaluation müssen auch ohne diese Oberfläche möglich sein.

Für FF3 werden Inferenzzeit und Verarbeitungslatenz nach Abschnitt 2.7 getrennt gemessen. Das Ziel aus H2 ist $P99 \text{ der Verarbeitungslatenz} < S/f_s$. Zusätzlich sind Anzahl und Anteil der Fristüberschreitungen sowie Erfassungspausen zwischen Fenstern zu berichten.

4.3 Evaluation

Für FF2 werden die Kennzahlen aus Abschnitt 2.6 auf derselben Testmenge ermittelt. Anzahl der Aufnahmen, Klassenverteilung und Ergebnisse je Aufnahme sind anzugeben (A9). Für FF3 werden Modellgröße, CPU-Auslastung und Prozessspeicher einschließlich Bibliotheken und Vorverarbeitung auf derselben Zielhardware gemessen (A8). Messbedingungen sowie Software-, Code- und Modellstand müssen nachvollziehbar sein (A5). Bei einer Modellkonvertierung sind numerische Abweichungen und Entscheidungen gegenüber dem Ausgangsmodell zu prüfen; die Prüfdaten und Akzeptanzgrenzen werden vor der Testauswertung festgelegt (A6). Wird die Live-GUI umgesetzt, ist ihr zusätzlicher Einfluss auf Latenz und Ressourcenbedarf bei ansonsten gleichen Bedingungen mit und ohne Anzeige zu messen.

Für FF4 wird eine dokumentierte Normalbedingung gezielt verändert. Vorverarbeitung, Modell und Schwelle bleiben dabei unverändert. Verglichen werden die Fehlalarmraten unter der ursprünglichen und der veränderten Bedingung. Eine Nachkalibrierung ist getrennt auszuwerten.

4.4 Kernanforderungen und Nachweis

Tabelle 4-1 enthält die Anforderungen A1–A13 mit Art, Priorität und Nachweis. F kennzeichnet funktionale Anforderungen an Systemfunktionen. NF bezeichnet nichtfunktionale Anforderungen an Nachvollziehbarkeit, Zeitverhalten und Betriebsstabilität. Methodische Anforderungen (M) betreffen Versuchsplanung und wissenschaftliche Auswertung. MUSS ist verbindlich; KANN bezeichnet die optionale Live-GUI. Ihr Nachweis ist nur bei Umsetzung erforderlich. Kapitel 6 dokumentiert Umsetzung und Prüfprotokoll. Kapitel 7 bewertet die Erfüllung und prüft die Hypothesen. Nichterfüllte technische Ziele und nicht bestätigte Hypothesen werden als Ergebnisse offengelegt.

Tabelle 4-1: Anforderungen, Prioritäten und Nachweise

ID	Art / Priorität	Anforderung	Nachweis
A1	F MUSS	Messdaten mit überprüfter Qualität erfassen	Kanäle, Einheiten und Zeitstempel prüfen; Bandbreite, Signal-Rausch- Abstand, Sättigung und Datenlücken anhand vorab festgelegter Grenzen bewerten
A2	M MUSS	Datenbasis nach Aufnahmen trennen	Ganze Aufnahmen zuordnen; Training und Kalibrierung nur mit Normaldaten; Vorverarbeitung nur an Trainingsdaten anpassen
A3	M MUSS	Verfahren einheitlich vergleichen	Gleiche Testfenster und Labels; gemeinsame Kalibrierungsregel und dokumentierte Eingaben
A4	F MUSS	Automatisch und lokal auswerten	Versuchslauf ohne Cloud-Dienst; Score, Schwelle und Entscheidung je gültigem Fenster; ungültige Fenster kennzeichnen
A5	NF MUSS	Daten und Softwarestände nachvollziehbar zuordnen	Lauf, Fenster, Code, Modell, Vorverarbeitung sowie Softwareabhängigkeiten und Versionen maschinenlesbar dokumentieren
A6	M MUSS	Zielkompatibilität und ausgeführtes Modell prüfen	Modelloperationen auf der Zielhardware prüfen; bei Konvertierung Scores und Entscheidungen nach vorab festgelegten Akzeptanzgrenzen vergleichen
A7	NF MUSS	Latenzziel aus H2 erfüllen: $P99 < S/f_s$	Inferenzzeit und Verarbeitungslatenz getrennt erfassen; Latenzverteilung, Fristüberschreitungen und Erfassungspausen berichten
A8	NF MUSS	Ohne Speicherabbruch und wachsenden Rückstand laufen	Modellgröße, Prozessspeicher, CPU und Rückstand über dokumentierte Messdauer erfassen; Aufwärmphase und Hintergrundlast angeben
A9	M MUSS	Erkennungsqualität bewerten	Konfusionsmatrix, Precision, Recall, F1, Fehlalarmrate und Accuracy auf derselben Testmenge; Ergebnisse je Aufnahme
A10	M MUSS	Geänderte Normalbedingungen prüfen	Fehlalarmraten bei unveränderter Vorverarbeitung, unverändertem Modell und unveränderter Schwelle vergleichen

ID	Art / Priorität	Anforderung	Nachweis
A11	M MUSS	Reproduzierbaren Testaufbau realisieren	Aufbau und Protokoll dokumentieren; sicheren Betrieb herstellen; Normal- und Anomaliezustände wiederholen; Rücksetzbarkeit und kontrollierte Veränderung prüfen
A12	F KANN	Live-GUI zur Ergebnisanzeige entwickeln	Angezeigte Werte mit gespeicherten Ergebnissen abgleichen; veraltete und fehlende Daten prüfen; Zusatzaufwand messen
A13	F MUSS	Messläufe starten und geordnet beenden	Start und Stopp prüfen; eindeutige Laufkennung und Erhalt vollständig erfasster Daten und Ergebnisse nachweisen

Kapitel 5 leitet aus A1–A13 Kriterien für Prüfstand, Messkette, Verarbeitung und Evaluation ab. MUSS-Prioritäten sind keine Gewichte: Bekannte Nichterfüllungen dürfen nicht durch Vorteile anderer Kriterien ausgeglichen werden. Noch offene Nachweise begrenzen die Vorauswahl. Die konkrete Lösung wird erst im Variantenvergleich festgelegt.

5 Variantenvergleich und Systementscheidung

Dieses Kapitel begründet die Systemauswahl anhand der Anforderungen A1–A13 und beantwortet damit den zweiten Teil von FF1. Zunächst wird der Prüfstand ausgewählt. Darauf folgen Sensorik und Edge-Plattform, Datenrepräsentation und Verfahren sowie die Entwicklungs- und Inferenzumgebung. Die abschließende Systementscheidung verbindet diese Teilauswahlen mit den noch ausstehenden Nachweisen.

Die folgende Darstellung rekonstruiert die Systementscheidung und die dabei festgelegten Prüfvorbehalte. Die Anforderungen beschreiben den ursprünglichen Zielmaßstab. Ob die jeweiligen Nachweise im vorhandenen Datenstand erbracht wurden, bewertet Kapitel 7.

5.1 Bewertungsrahmen und Vorgehen

Die Bewertung berücksichtigt die Eignung für die Forschungsfragen, die technische Umsetzbarkeit und den erforderlichen Zusatzaufwand. Anforderungen und Auswahlgründe müssen dabei nachvollziehbar zusammenhängen (NASA, 2023). Für den Prüfstand stehen wiederholbare Zustände und kontrollierte Veränderungen im Vordergrund (A10, A11). Für die Messkette sind Signalqualität und zeitliche Erfassung entscheidend (A1). Hardware und Software müssen die lokale Verarbeitung sowie ihre überprüfbare Ausführung ermöglichen (A4–A8).

Zuerst werden die MUSS-Anforderungen geprüft. Eine bekannte Nichterfüllung kann nicht durch Punkte in anderen Kriterien ausgeglichen werden. Noch offene Nachweise, etwa zur Signalqualität oder Latenz, führen zu einer vorläufigen Auswahl mit Prüfvorbehalt. Sicherheitsmaßnahmen sind vor dem Versuch umzusetzen. Erst unter diesen Voraussetzungen werden geeignete Entwurfsvarianten gegeneinander abgewogen.

Für Prüfstand, Edge-Plattform und Software werden gewichtete Nutzwertmatrizen verwendet. Die Punkteskala reicht von 0 bis 4; ein höherer Wert bedeutet einen höheren Nutzen für dieses Projekt. Die Kriterientabellen erläutern die Bewertungsmaßstäbe. Zwischenwerte 1 und 3 kennzeichnen abgestufte Einschätzungen zwischen den beschriebenen Ankern. Gleich große Punktabstände werden vereinfachend als gleich große Nutzenzuwächse behandelt. Das ist eine Modellannahme und keine gemessene Leistungsrelation.

Die Gewichte beziehen sich auf den Unterschied zwischen dem ungünstigsten und dem günstigsten beschriebenen Zustand eines Kriteriums. Je größer dessen Bedeutung für die Forschungsfragen, desto höher wird dieser Unterschied gewichtet. Dieses Vorgehen orientiert sich am Prinzip der Swing-Gewichtung (Department for Communities and Local Government, 2009, S. 63–65). Die konkreten Gewichte und Punkte sind eigene Entwurfsannahmen; sie stammen weder aus Herstellerangaben noch aus einer Expertenbefragung. Jede Matrix wird für sich auf 100 % normiert. Nutzwerte verschiedener Auswahlbereiche sind nicht miteinander vergleichbar.

$$N_j = \sum_{i=1}^m \frac{w_i}{100} \cdot s_{ij}$$

Dabei ist m die Anzahl der Kriterien, N_j der Nutzwert der Alternative j , w_i das in Prozent angegebene Gewicht des Kriteriums i und s_{ij} dessen Punktwert. Der Nutzwert liegt zwischen 0 und 4. Er ist weder eine Erfolgswahrscheinlichkeit noch ein Nachweis erfüllter Anforderungen.

Die Sensitivität wird rechnerisch geprüft: Jedes Gewicht wird einzeln um fünf Prozentpunkte erhöht und gesenkt; die übrigen Gewichte werden proportional auf zusammen 100 % angepasst. Zusätzlich werden gleiche Gewichte und ausgewählte Änderungen unsicherer Punktwerte um einen Punkt betrachtet. Diese Prüfungen zeigen die Abhängigkeit von den Annahmen, ersetzen aber keine Messung (NASA, 2023).

5.2 Bewertung der Prüfstandsalternativen

Der Prüfstand muss normale und gezielt veränderte Zustände wiederholt herstellen können (A11). Für FF4 muss außerdem eine Normalbedingung veränderbar sein, ohne diese Änderung bereits als Anomalie zu kennzeichnen (A10). Tabelle 5-1 vergleicht hierfür drei Alternativen anhand ihrer Kontrollierbarkeit, des zusätzlichen Aufwands und ihrer Aussagegrenzen.

Tabelle 5-1: Bewertung der Prüfstandsalternativen

Alternative	Kontrollierbarkeit und Wiederholung	Zusatzaufwand und Aussagegrenze
Lüfter mit PWM-Ansteuerung	Betriebspunkte vorgebar; Versuchszustände bei fester Montage wiederholbar zu gestalten	Wenige Antriebskomponenten; Lastvariation und Übertragbarkeit auf Industriemaschinen begrenzt
Servomotor mit Regler	Geregelte Drehzahl- und Bewegungsprofile; Lastvariation mit zusätzlicher Belastungseinrichtung	Regler, mechanische Kopplung und gegebenenfalls Lastmaschine nötig; Regelung beeinflusst das Schwingungsverhalten
Motor-/ Lagerprüfstand	Drehzahl, Last und Lagerzustände gezielt untersuchbar	Zusätzliche Mechanik, Messtechnik und Schutzmaßnahmen; größere Nähe zur Lagerdiagnostik

Quelle: Eigene Bewertung auf Grundlage von ARCTIC (o. J.), Oriental Motor (o. J.) und Universität Paderborn (o. J.). Der Aufwand ist eine qualitative Einschätzung der benötigten Komponenten.

Für die Bewertung wird vorausgesetzt, dass bei allen drei Varianten sichere, rücksetzbare Versuchszustände realisiert werden können. Dies bleibt nach A10 und A11 am fertigen Aufbau nachzuweisen. Die folgenden Kriterien unterscheiden die verbleibenden Gestaltungsmöglichkeiten. Der Realisierungsaufwand erhält das größte Gewicht, weil die Arbeit den Methodenvergleich und keine Entwicklung einer komplexen Antriebsanlage untersucht. Es werden benötigte Baugruppen eingeschätzt; Kosten oder Arbeitsstunden werden nicht behauptet.

Tabelle 5-2: Kriterien und Gewichte für den Prüfstand

Kriterium	Gewicht	Bewertungsmaßstab und Anforderungsbezug
P1 Betriebspunkte einstellen	25 %	A10, A11. 0: keine Vorgabe; 2: einzelne Stellgröße ohne unabhängige Lastregelung; 4: geregelte Drehzahl und zusätzlich Last oder Bewegung vorgebar.
P2 Einflüsse trennen	25 %	A10, A11. 0: Änderungen nicht zuordenbar; 2: mehrere gekoppelte Einflüsse; 4: einfache Variation bei fester Mechanik und wenigen Einflussgrößen.
P3 Aufbauaufwand	40 %	A11. 0: im Projekt nicht realisierbar; 2: Antrieb, Regler und Kopplung aufzubauen; 4: integrierter Antrieb mit wenigen Zusatzbaugruppen.
P4 Anwendungsnähe	10 %	FF1, FF4. 0: kein rotierendes System; 2: einfache rotierende Baugruppe; 4: gezielt untersuchbare Maschinen- und Lagerzustände.

Quelle: Eigene Kriterien und Gewichte, abgeleitet aus Kapitel 4. Maßstäbe beschreiben projektbezogene Präferenzen.

Tabelle 5-3: Gewichtete Bewertung der Prüfstandsalternativen

Alternative	P1	P2	P3	P4	Nutzwert
Lüfter mit PWM	2	4	4	2	3,30
Servomotor mit Regler	4	3	2	3	2,85
Motor-/Lagerprüfstand	4	3	1	4	2,55

Quelle: Eigene Punktvergabe auf Grundlage des Variantenvergleichs und der vorausgehenden Kriterien; keine experimentellen Messwerte.

Der Lüfter erhält weniger Punkte bei der Betriebspunktregelung und der Nähe zur Maschinenüberwachung, bietet aber einen einfachen Aufbau mit wenigen zu kontrollierenden Einflussgrößen. Beim Servo kommen Rückführung und mechanische Kopplung hinzu. Der Motor-/Lagerprüfstand ermöglicht die gezielteste Lageruntersuchung, benötigt jedoch zusätzliche Belastungs- und Messbaugruppen. Die Punkte bewerten diese angenommenen Aufbauvarianten; sie belegen keine schlechtere Reproduzierbarkeit eines professionellen Prüfstands.

Unter den gewählten Gewichten wird der Lüfter mit 3,30 Punkten bevorzugt. Beispielhaft ergibt sich $0,25 \cdot 2 + 0,25 \cdot 4 + 0,40 \cdot 4 + 0,10 \cdot 2 = 3,30$. In den acht einzelnen Gewichtsänderungen bleibt er vorn. Bei gleichen Gewichten erreichen dagegen alle Varianten 3,00 Punkte. Die Auswahl hängt damit wesentlich vom begrenzten Realisierungsaufwand ab. Eine einzelne Änderung eines Punktwerts um einen Punkt kehrt die Ausgangsrangfolge an der Spitze nicht um; gemeinsame Änderungen können dies tun.

Für den Kernversuch wird deshalb der Lüfter gewählt. Ein Servomotor wäre bei Lastwechseln oder dynamischen Bewegungsprofilen sinnvoll, ein Motor-/Lagerprüfstand bei konkreten Lagerfehlern. Diese Aufgaben gehören nicht zum Kernvergleich. Die Ergebnisse bleiben auf den untersuchten Versuchsaufbau begrenzt.

Als konkrete Variante wurde der ARCTIC P12 Pro PST vorgesehen, dessen Datenblatt PWM-Ansteuerung und ein Drehzahlsignal ausweist (ARCTIC, o. J.). Ein PWM-Tastgrad ist eine Stellgröße und kein Nachweis einer konstanten Drehzahl. Die Konzeption sah deshalb die Erfassung der tatsächlichen Drehzahl vor. Dieser Nachweis wurde bisher nicht erbracht; die Betriebspunkte werden durch ihre dokumentierten PWM-Vorgaben beschrieben. Für FF4 wurde in der früheren Prüfreihe ein zweiter normaler Betriebspunkt bei unveränderter Montage untersucht; diese Werte bleiben im Anhang erhalten. Den aktuellen Handyaufbau und Messablauf beschreibt Kapitel 6, die zugehörigen Befunde und Grenzen Kapitel 7.

5.3 Auswahl von Sensorik und Edge-Plattform

Die Sensorik muss die vorgesehenen dreiachsigen Beschleunigungssignale mit ausreichender Qualität erfassen. Verglichen werden zwei digital anschließbare MEMS-Sensoren und eine industrielle Messkette. Tabelle 5-4 nennt Eigenschaften, die für A1 relevant sind. Datenblattwerte belegen noch keine ausreichende Signalqualität am späteren Montageort.

Tabelle 5-4: Vergleich der Sensorik und Messketten

Alternative	Relevante Herstellerangaben	Bewertung für den Prüfstand
ADXL345	Drei Achsen; $\pm 2/4/8/16$ g; I ² C/SPI; FIFO	Direkte digitale Erfassung; Bandbreite, Auflösung und Rauschen am Aufbau prüfen
ADXL355	Drei Achsen; $\pm 2/4/8$ g; 20-Bit-ADC; typische Rauschdichte $22,5 \mu\text{g}/\sqrt{\text{Hz}}$ bei ± 2 g	Alternative bei höheren Anforderungen an Auflösung und Rauschen; eigene Integrationsprüfung nötig
PCB 356A32 mit IEPE-Messkette	Drei Achsen; ± 50 g; 100 mV/g ; $1\text{--}4000 \text{ Hz}$ innerhalb $\pm 5 \%$	Größerer spezifizierter Frequenzbereich; Konstantstromspeisung und externe A/D-Wandlung nötig

Quelle: Eigene Einordnung nach Analog Devices (2022, 2025) und PCB Piezotronics (o. J.). Die Zahlen sind Herstellerangaben.

Für den ersten Systementwurf wird der ADXL345 mit digitaler I²C-Anbindung vorgesehen. Er bietet eine vollständige dreiachsige Messung ohne externe A/D-Wandlung. Auch der ADXL355 erfüllt dieses Integrationsprinzip; die Wahl des ADXL345 ist daher eine zu prüfende Ausgangsentscheidung und kein Nachweis seiner Überlegenheit. Reichen Auflösung oder Signal-Rausch-Abstand nicht aus, wird der ADXL355 als Alternative geprüft. Eine IEPE-Messkette ist erst bei entsprechendem Bandbreiten- oder Messqualitätsbedarf zu rechtfertigen.

Die Pilotmessung prüft effektive Erfassungsrate, neue Sensorwerte, Datenlücken, Sättigung und den Abstand des Nutzsignals zum Rauschboden bei dokumentierter Montage (A1). Die Akzeptanzgrenzen sind vor der abschließenden Testserie festzulegen. Beim ADXL345 beträgt die maximal empfohlene Sensorausgaberate bei 400-kHz-I²C 800 Hz; bei 100-kHz-I²C sind es 200 Hz. Diese Grenzen ersetzen keine Messung der tatsächlich erreichten Erfassung (Analog Devices, 2022, S. 17).

Die Edge-Plattform muss neben der Inferenz auch Datenerfassung, Vorverarbeitung, Baselines und Messwerkzeuge tragen (A4, A7, A8). Tabelle 5-5 vergleicht deshalb den Ressourcenrahmen und die daraus abgeleiteten Integrationsfolgen. Taktraten und Beschleunigerangaben erlauben keinen unmittelbaren Vergleich der Inferenzzeiten.

Tabelle 5-5: Bewertung der Edge-Plattformen

Alternative	Dokumentierter Ressourcenrahmen	Bewertung für den Methodenvergleich
STM32H743	Cortex-M7 bis 480 MHz; bis 2 MB Flash und 1 MB RAM	Hardwarenahe Umsetzung möglich; knapper Speicher und eigenständige Portierung der Pipeline
Raspberry Pi 4 Model B	Vier Cortex-A72-Kerne mit 1,8 GHz; RAM im GB-Bereich	Plausible Linux-Plattform für CPU-Inferenz und Messwerkzeuge
Raspberry Pi 5	Vier Cortex-A76-Kerne mit 2,4 GHz; RAM im GB-Bereich	Nominell größere CPU-Reserve; konkrete Latenz und Speicherbelegung bleiben zu messen
Jetson Orin Nano Super	Sechs Cortex-A78AE-Kerne; 8 GB RAM; GPU-Beschleunigung	Sinnvoll bei GPU-bedürftigen Modellen; Nutzen für den vorgesehenen CPU-Vergleich nicht belegt

Quelle: Eigene Bewertung nach STMicroelectronics (2026), Raspberry Pi Ltd. (o. J., 2026) und NVIDIA (o. J.).

Die Ressourcenangaben werden nicht in erwartete Inferenzzeiten umgerechnet. Die gewichtete Vorauswahl verbindet den nominellen Ressourcenrahmen mit der Integration der vollständigen Pipeline. Das Gewicht für zeitliche Kontrollierbarkeit bleibt begrenzt, weil H2 ein empirisches Latenzziel und keine harte Echtzeitgarantie formuliert.

Tabelle 5-6: Kriterien und Gewichte für die Edge-Plattform

Kriterium	Gewicht	Bewertungsmaßstab und Anforderungsbezug
E1 Nomineller Ressourcenrahmen	35 %	A8. 0: keine plausible Reserve; 2: eingeschränkter Rahmen; 4: GB-Arbeitsspeicher und größere nominelle CPU-Reserve im Variantenfeld. Kein Laufzeitnachweis.
E2 Integration der Gesamtpipeline	35 %	A4–A6. 0: keine geeignete Umgebung; 2: erhebliche Anpassungen; 4: allgemeine Linux-/Python-Umgebung für Erfassung, Verfahren und Messwerkzeuge.
E3 Zeitliche Kontrollierbarkeit	15 %	A7. 0: Ablauf kaum beeinflussbar; 2: allgemeines Betriebssystem mit schwankender Last; 4: hardwarenah planbare Ausführung. Keine Fristgarantie.
E4 Passung zum CPU-Kernvergleich	15 %	FF1, FF3. 0: ungeeignete Plattformklasse; 2: zusätzliche Beschleuniger bleiben ungenutzt oder die Plattform erzwingt einen Portierungsfokus; 4: CPU-System ohne solche Zusatzschwerpunkte.

Quelle: Eigene Kriterien und Gewichte aus Kapitel 4. Ressourcen, Softwareintegration, Zeitplanung und Untersuchungsumfang werden getrennt betrachtet.

Tabelle 5-7: Gewichtete Bewertung der Edge-Plattformen

Alternative	E1	E2	E3	E4	Nutzwert
STM32H743	1	1	4	2	1,60
Raspberry Pi 4	3	4	2	4	3,35
Raspberry Pi 5	4	4	2	4	3,70
Jetson Orin Nano Super	4	4	2	2	3,40

Quelle: Eigene Punktvorgabe nach dem dokumentierten Ressourcenrahmen und den Kriterien E1-E4; keine gemessenen Leistungswerte.

Der STM32 wird bei Ressourcen und Integration wegen des begrenzten Speichers und der notwendigen Portierung niedrig bewertet. Die Linux-Plattformen erhalten bei der allgemeinen Integration denselben Wert. Der Pi 4 erhält für E1 einen Punkt weniger als der Pi 5; dies bewertet ausschließlich die nominelle CPU-Reserve. Die Jetson-GPU bringt für den hier abgegrenzten CPU-Vergleich keinen zusätzlichen Untersuchungsnutzen. Ihre Leistungsfähigkeit wird damit nicht infrage gestellt.

Der Raspberry Pi 5 erreicht 3,70 Punkte und wird als Zielplattform gewählt. Er bleibt bei den acht einzelnen Gewichtsänderungen und bei gleichen Gewichten vorn. Die Einschätzung ist jedoch punktempfindlich: Wird seine Ressourcenbewertung E1 um einen Punkt gesenkt, liegt er mit 3,35 gleichauf mit dem Pi 4 und unter dem Jetson mit 3,40. Die nominelle Reserve ist deshalb eine Entwurfsbegründung, kein Leistungsbeleg. Die Erfüllung von A7 und A8 wird an der vollständigen Pipeline gemessen; ein zusätzlicher Hardwarebenchmark gehört nicht zum Kernversuch.

5.4 Datenbasis, Signalrepräsentation und Verfahren

Für die Evaluation werden eigene reale Messläufe gewählt. Damit umfasst die Untersuchung die vollständige Kette vom Sensor bis zur Entscheidung. Öffentliche Datensätze können die Methodenentwicklung unterstützen, ersetzen jedoch keinen Nachweis an der gewählten Messkette. Simulierte Daten eignen sich für Funktionstests, belegen aber keine Erkennungsleistung am realen Prüfstand. Ganze Aufnahmen werden nach A2 getrennt zugeordnet. Training und Schwellenwertkalibrierung verwenden ausschließlich Normaldaten.

Tabelle 5-8: Bewertung der Signalrepräsentationen

Darstellung	Informationsgehalt und Grenze	Vorgesehene Rolle
Dreiaxsiges Zeitfenster	Erhält Reihenfolge und Achsenbeziehungen; höhere Eingabedimension	Gemeinsame Ausgangsdaten für den Hauptvergleich
Zeitbereichskennwerte	Kompakt und interpretierbar; zeitliche Struktur wird verdichtet	RMS als statistische Baseline

Darstellung	Informationsgehalt und Grenze	Vorgesehene Rolle
Frequenzspektrum mittels FFT	Zeigt Frequenzanteile; abhängig von Abtastung, Fensterdauer und Fensterfunktion	Ergänzende Analyse gespeicherter Pilotmessungen
Zeit-Frequenz-Darstellung	Zeigt zeitlich wechselnde Frequenzanteile; zusätzliche Parameter und Rechenaufwand	Keine zusätzliche Variante im Kernvergleich

Quelle: Eigene Einordnung auf Grundlage von Ruff et al. (2021) und SciPy developers (o. J. a, o. J. b).

Die Fast Fourier Transform (FFT) ist ein effizienter Algorithmus zur Berechnung der diskreten Fourier-Transformation. Sie wird ergänzend eingesetzt, um beobachtbare Frequenzanteile in Pilotmessungen zu untersuchen und die Wahl von Erfassungsrate und Fensterdauer zu unterstützen. Für H gleichmäßig abgetastete Werte beträgt der Abstand der Frequenzstützstellen $\Delta f = f_s/H$; dies allein beschreibt noch nicht die Trennbarkeit benachbarter Frequenzanteile (SciPy developers, o. J. a, o. J. b).

Eine FFT behebt weder Datenlücken noch Aliasing. Vor ihrer Auswertung müssen zeitliche Regelmäßigkeit, nutzbare Sensorbandbreite und mögliche Frequenzüberlagerungen durch die Abtastung geprüft werden. Mittelwertbehandlung, Fensterfunktion und Amplitudenskalierung sind zu dokumentieren. Die Spektralanalyse erfolgt ergänzend außerhalb der Live-Pipeline. FFT-Merkmale würden eine weitere Verfahrensvariante bilden und müssten vor der Testauswertung festgelegt sowie mit ihrem Zusatzaufwand bewertet werden. Die FFT selbst liefert keine Anomalieentscheidung.

Tabelle 5-9: Auswahl der Anomalieverfahren

Verfahren	Rolle und Auswahlgrund	Wesentliche Grenze
Autoencoder	Neurales Hauptverfahren; prüft den Nutzen einer erlernten multivariaten Rekonstruktion	Eine gute Trennung der Zustände ist empirisch zu prüfen
RMS mit Schwelle	Statistische Minimalbaseline; transparente Verdichtung auf einen Kennwert	Verwirft die zeitliche Reihenfolge
Isolation Forest	Klassische Lernbaseline; ergänzt den RMS um eine multivariate Bewertung	Nutzt keine explizite zeitliche Nachbarschaft
One-Class SVM	Plausible Alternative zur klassischen Lernbaseline	Zusätzliche Kernel- und Hyperparameterwahl; keine weitere Vergleichsrolle
Überwachte Klassifikation	Geeignet bei repräsentativen gelabelten Fehlerklassen	Benötigt Fehlerbeispiele im Training und passt damit nicht zum gewählten Szenario

Quelle: Eigene Rollenverteilung nach Liu et al. (2008), Ruff et al. (2021) und scikit-learn developers (o. J. d).

Der Hauptvergleich umfasst Autoencoder, RMS und Isolation Forest. So werden die in Kapitel 1 festgelegten drei Verfahrensklassen abgedeckt. Alle erhalten dieselben $H \times 3$ -

Ausgangsfenster. Die Standardisierung wird ausschließlich an Trainingsdaten angepasst. Der Autoencoder rekonstruiert das standardisierte Fenster, der Isolation Forest verwendet dessen Werte in fester Reihenfolge als Vektor. Die RMS-Baseline berechnet einen gemeinsamen Effektivwert über alle 3H standardisierten Werte; dieser Score ist dimensionslos und vom RMS der Beschleunigung in g zu unterscheiden.

Für jedes Verfahren wird die Schwelle als empirisches 99. Perzentil seiner Scores auf separaten normalen Validierungsaufnahmen bestimmt. Größere Scores bedeuten stets eine stärkere Auffälligkeit. Die genaue Quantilsberechnung, Modellparameter und Fensterbildung werden vor der Testauswertung festgehalten (A2, A3). Eine bessere Erkennungsqualität des Autoencoders wird damit nicht vorausgesetzt, sondern durch H1 geprüft.

5.5 Kriterienbasierte Bewertung der Softwarealternativen

Die Softwareauswahl betrifft den Entwicklungs- und Ausführungspfad des Autoencoders. TensorFlow/Keras und PyTorch dienen zur Modellbildung; TFLite/LiteRT, ExecuTorch und ONNX Runtime bieten Wege zur lokalen Inferenz. Der Prüfstand wurde bereits in Abschnitt 5.2 bewertet. Die nachfolgenden Kriterien betreffen die Software und werden aus A4–A8 abgeleitet.

Tabelle 5-10: Kriterienbasierter Vergleich der Softwarealternativen

Entwicklungs- und Ausführungspfad	Übergang zur Inferenz	Lokale Ausführung	Integrations- und Prüfaufwand
TensorFlow/Keras → TFLite/LiteRT	Dokumentierter Konverter für Keras-Modelle; .tflite-Datei	CPU-Inferenz mit passender Laufzeit	Modelloperationen, Paketkompatibilität und Konvertierung prüfen
PyTorch → ExecuTorch	Modellexport und Anpassung an ein Backend; .pte-Datei	Arm-CPU über geeignetes Backend, z. B. XNNPACK	Export, Backend und numerische Übereinstimmung prüfen
PyTorch → ONNX Runtime	Export in das ONNX-Austauschformat	CPU-Paket für Linux ARM64	Exporter, ONNX-Operatorversionen und Laufzeit abstimmen
TensorFlow/Keras → TensorFlow direkt	Kein Wechsel des Modellformats erforderlich	TensorFlow-CPU-Paket für Linux ARM64	Paketkompatibilität und Ressourcenbedarf der vollständigen Umgebung prüfen

Quelle: Eigene Bewertung nach Google (2026a, 2026c, 2026d), PyTorch (o. J.) und ONNX Runtime developers (o. J.). Die Zielkompatibilität des konkreten Modells ist bei jeder Alternative zu prüfen.

Die gemeinsame Voraussetzung ist, dass Modelloperationen, Python-Version und Betriebssystem auf dem Zielgerät zusammenpassen (A6). Dies ist für das konkrete Modell noch nachzuweisen. Dokumentierte ARM64-Unterstützung allein bestätigt keine beliebige Raspberry-Pi-OS-Konfiguration. Für TFLite wird ein separates Interpreter-Paket beschrieben; Modelle mit zusätzlichen TensorFlow-Operationen können eine umfangreichere Umgebung benötigen (Google, 2026c). Diese Einschränkung wird bei der Modellkonvertierung geprüft.

Tabelle 5-11: Kriterien und Gewichte für die Softwareauswahl

Kriterium	Gewicht	Bewertungsmaßstab und Anforderungsbezug
S1 Entwicklung und Inferenz trennen	30 %	A5, A8. 0: kein getrennt ladbares Modell; 2: gespeichertes Modell benötigt vollständiges Framework; 4: separates Inferenzpaket vorgesehen.
S2 Modellübergang nachvollziehen	25 %	A5, A6. 0: kein dokumentierter Weg; 2: nur teilweise dokumentiert; 4: offizieller Weg zum Speichern bzw. Exportieren und Laden.
S3 Python-Anbindung	25 %	A4, A5. 0: keine Anbindung; 2: eigene Brücke nötig; 4: offiziell dokumentierte Python-API.
S4 Linux-ARM64-CPU unterstützen	20 %	A4, A6. 0: kein dokumentierter Weg; 2: eigene Portierung nötig; 4: dokumentiertes CPU-Paket für Linux ARM64. Konkrete Versionen bleiben zu prüfen.

Quelle: Eigene Kriterien und Gewichte aus Kapitel 4. Die Punkte bewerten dokumentierte Wege, nicht deren gemessenen Ressourcenbedarf.

Tabelle 5-12: Gewichtete Bewertung der Softwarealternativen

Entwicklungs- und Ausführungspfad	S1	S2	S3	S4	Nutzwert
TensorFlow/Keras → TFLite/LiteRT	4	4	4	4	4,00
PyTorch → ExecuTorch	4	4	4	4	4,00
PyTorch → ONNX Runtime	4	4	4	4	4,00
TensorFlow/Keras → TensorFlow direkt	2	4	4	4	3,40

Quelle: Eigene Bewertung der offiziellen Dokumentation von Google (2026a, 2026c, 2026d), PyTorch (o. J.) und ONNX Runtime developers (o. J.).

Die drei separaten Inferenzwege erreichen jeweils 4,00 Punkte. Sie bieten die in S1–S4 beschriebenen Möglichkeiten. Die direkte TensorFlow-Ausführung erhält bei S1 zwei Punkte, weil dabei das vollständige Framework vorgesehen ist; dafür entfällt der Formatwechsel. Eine hohe Punktzahl sagt nichts über spätere Latenz, Speichernutzung oder Erkennungsqualität aus. Diese Größen werden nicht mit unbelegten Schätzwerten bewertet.

Der Gleichstand bleibt bei gleichen Gewichten und bei allen acht einzelnen Gewichtsänderungen bestehen. Sinkt ein bisher mit vier Punkten bewertetes Kriterium einer Alternative um einen Punkt, verliert diese ihren Gleichstand an der Spitze. Die Matrix zeigt daher mehrere geeignete Wege und keinen eindeutigen TensorFlow-Vorsprung. Bei gleich geeigneten Alternativen ist eine begründete Entwurfsentscheidung zulässig (NASA, 2023).

Für diese Arbeit wird aus dieser Gruppe TensorFlow/Keras mit einem Float32-TFLite-Modell gewählt. Der dokumentierte direkte Konverter für Keras-Modelle ermöglicht einen klaren Weg von der Modellbildung zur gespeicherten Inferenzdatei und deren Prüfung gegen das Ausgangsmodell (Google, 2026a). Damit wird eine konkrete Umsetzung innerhalb der gleich

bewerteten Gruppe festgelegt. ExecuTorch bleibt eine gleichwertige Entwurfsalternative nach den verwendeten Maßstäben; ONNX Runtime wäre besonders bei einem gewünschten Austauschformat zweckmäßig. Ein solcher Austausch ist keine eigene Anforderung des Kernversuchs.

Im Hauptvergleich wird auf Quantisierung verzichtet. So wird zunächst keine zusätzliche Änderung der Zahlendarstellung untersucht. Die Dokumentation empfiehlt ein nicht quantisiertes TFLite-Modell als Ausgangspunkt für die Kompatibilitätsprüfung (Google, 2026b). Ob die ausgewählte Laufzeit A7 und A8 erfüllt, entscheidet erst die Messung. Auch eine kleine Modelldatei garantiert keinen kleinen Prozessspeicher. Konkrete Pakete, Versionen und ihre Einbindung werden in Kapitel 6 dokumentiert.

5.6 Systementscheidung und Übergang zur Implementierung

Die abschließende Tabelle fasst die Entscheidungen und die jeweils noch erforderlichen Nachweise zusammen. Die Nutzwerte unterstützen die Auswahl innerhalb des jeweiligen Variantenfelds. Sie sind an die offengelegten Gewichte, Punkte und Prüfvorbehalte gebunden. Die praktische Eignung bleibt Gegenstand von Implementierung und Evaluation.

Tabelle 5-13: Systementscheidungen und ausstehende Nachweise

Bereich	Auswahl	Anforderungsbezug und Nachweis
Prüfstand	Lüfter mit PWM; 3,30 Punkte	A10, A11: wiederholbare Zustände und zweiter normaler Betriebspunkt
Sensorik	ADXL345 als Ausgangsvariante; ADXL355 als Alternative	A1: Signalqualität und zeitliche Erfassung vor der Testserie prüfen
Edge-Plattform	Raspberry Pi 5; 3,70 Punkte	A4, A7, A8: vollständige Pipeline und Zeitbudget messen
Datenbasis	Eigene reale Messläufe; Normaldaten für Training und Kalibrierung	A2, A3, A9: getrennte Aufnahmen und gemeinsame Testbasis
Verfahren	Autoencoder, RMS und Isolation Forest	A3, A9: gleiche Fenster und Kalibrierungsregel; Erkennungsqualität vergleichen
FFT	Ergänzende Analyse der Pilotmessungen	A1: Frequenzanteile innerhalb der überprüften Messbandbreite untersuchen
Neuronale Software	TensorFlow/Keras → Float32-TFLite; 4,00 Punkte, Gleichstand mit zwei Alternativen	A5–A8: Konvertierung, Zielkompatibilität und Ressourcen prüfen
Bedienung und Ausgabe	Start/Stop und gespeicherte Ergebnisse; Live-GUI optional	A4, A5, A12, A13: Datenzuordnung, geordnetes Beenden und gegebenenfalls Anzeige prüfen

Quelle: Eigene Zusammenführung der Entscheidungen aus den Abschnitten 5.2–5.5.

6 Implementierung und aktueller Versuchsaufbau

6.1 Ziel und Abgrenzung des Handyversuchs

Der aktuelle Vergleich untersucht, wie RMS, Isolation Forest und Autoencoder auf eine zusätzliche, über den Tisch eingeleitete Vibration reagieren. Die Anregung entsteht durch einen Anruf auf einem auf dem Tisch liegenden Handy. Der Lüfter bleibt am Betriebspunkt 75 % PWM. Eine Platte vor dem Luftauslass gehört nicht zu diesem Versuch. Die Handyvibration ist eine äußere mechanische Anregung und kein nachgewiesener innerer Lüfter- oder Lagerschaden.

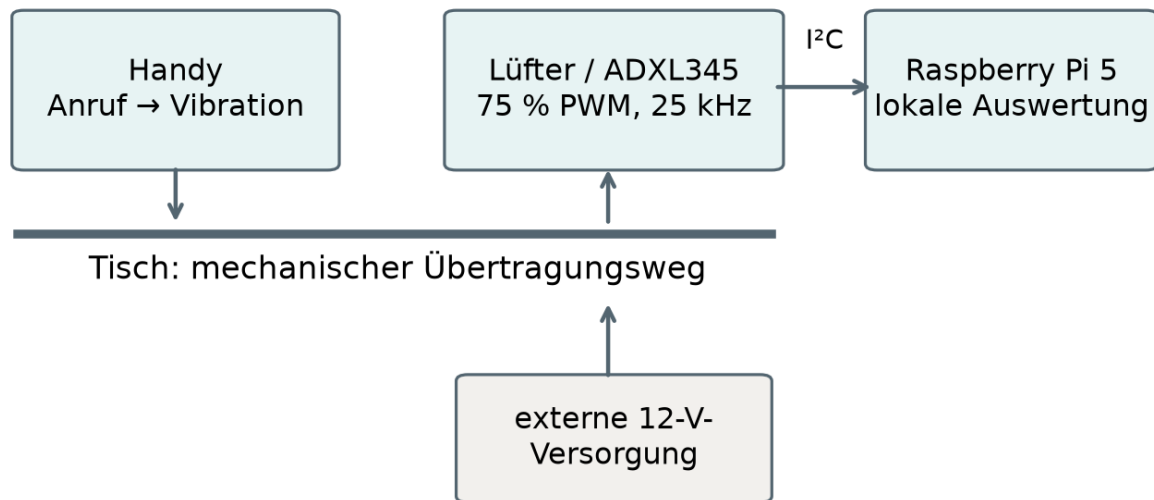
Die Hardwareauswahl aus Kapitel 5 bleibt bestehen. Geändert sind die Art der Anregung, das Versuchsprotokoll und der verwendete Software-/Modellstand. Die am 19. September gespeicherten Aufnahmen verwenden das Normalprofil `standlauf_pwm75_200hz` vom 18. September. Es ist nicht identisch mit dem früheren v4-Paket. Deshalb werden die früheren Erkennungs- und Ressourcenwerte nicht als Messungen des neuen Versuchs ausgegeben. Innerhalb der Handyreihe bleiben Modellartefakte, Scaler und Schwellen unverändert.

Die Untersuchung besitzt drei Ebenen: Einzelprozesse zeigen die jeweilige Methode ohne die beiden anderen Auswerter; ein Offline-Replay vergleicht alle Methoden auf denselben gespeicherten Rohfenstern; der gemeinsame Live-Lauf vergleicht zusätzlich die tatsächlichen Ausgabezeitpunkte auf einer gemeinsamen Aufnahme. Nur der letzte Modus verbindet identische aktuelle Sensoreingaben mit gleichzeitig aktiven Auswertern.

6.2 Hardware und mechanischer Übertragungsweg

Der Aufbau besteht aus einem Raspberry Pi 5, einem über I²C angeschlossenen ADXL345 und einem ARCTIC P12 Pro PST mit externer 12-V-Versorgung. Die zuvor beschriebene Befestigung des Sensors am feststehenden Lüfterrahmen bleibt die dokumentierte Hardwaregrundlage. Im aktuellen Profil ist die feste Montage als vom Nutzer bestätigt vermerkt. Eine neue metrische Vermessung der Orientierung oder der mechanischen Übertragungsfunktion liegt nicht vor.

Das Handy regt den Tisch an; die über den Aufbau übertragene Bewegung wird am ADXL345 erfasst. Für einen Vergleich zwischen getrennten Anrufen wären unveränderte Handyposition, Auflage, Vibrationsmuster und Umgebungsbedingungen erforderlich. Diese Gleichheit ist eine Versuchsannahme, keine durch zusätzliche Sensoren bestätigte Tatsache. Das vorbereitete Planungsdokument enthält keine vollständig bestätigte Handybeschreibung oder Geometrie. Das eingefrorene Paket ist deshalb als Entwicklungs-/Pilotstand zu behandeln. Im gemeinsamen Live-Lauf ist die Gleichheit der Eingaben dagegen unmittelbar durch den gemeinsamen Rohdatenstrom und Fensterhashes belegt.



Schema, kein Foto und nicht maßstäblich. Handyposition und Stärke nicht vermessen.

Abbildung 6-1: Schematischer Aufbau des Handyversuchs. Die Darstellung erklärt den vorgesehenen Übertragungsweg; sie ist kein Foto und bestätigt keine nicht vermessene Geometrie. Quelle: eigene Darstellung anhand von B12.

Tabelle 6-1: Hardware und Betriebszustand der Handyreihe

Bestandteil	Dokumentierter Stand	Grenze des Nachweises
Zielrechner	Raspberry Pi 5, ARM64, vier logische Kerne	Keine Aussage über andere Rechner
Lüfter	ARCTIC P12 Pro PST, externe 12 V	Keine kontinuierliche Spannungs- oder Drehzahlmessung
Sensor	ADXL345 am feststehenden Lüfterrahmen	Orientierung und Übertragungsfunktion nicht kalibriert
Stellanschluss	BCM-GPIO18, physischer Pin 12; Hardware-PWM	Rücklesung der Einstellung, keine elektrische Signalprüfung
Betriebspunkt	75 % Tastgrad bei 25 kHz	Keine Gleichsetzung mit 75 % Drehzahl
Anregung	Handyvibration auf dem Tisch durch Anruf	Beginn, Ende und Amplitude nicht unabhängig gemessen
Vergleichseinheit	128 aufeinanderfolgende XYZ-Punkte	Fenster sind innerhalb eines Ereignisses zeitlich abhängig

6.3 PWM, Sensor und Zeitbasis

Die protokollierte Hardware-PWM besitzt eine Periode von 40.000 ns, eine Einschaltdauer von 30.000 ns und enable = 1; daraus folgen 25 kHz und 75 % Tastgrad. Diese Einstellung wird vor den Aufnahmen zurückgelesen. Die Erkennung steuert den Lüfter nicht. Das Ende der Handyaufzeichnung ist daher nicht mit einem automatisch ausgeführten 0%-Befehl gleichzusetzen. Der Betriebspunkt wird in dieser Revision weder geändert noch aus einer Scoreüberschreitung neu berechnet.

Der ADXL345 arbeitet nominell mit 200 Hz, ± 2 g, voller Auflösung und FIFO-Stream-Erfassung. Der Umrechnungsfaktor beträgt 0,0039 g pro LSB. I²C-Bus 1 ist mit 100 kHz

konfiguriert. Registerrücklesungen und Qualitätsmerkmale sind in run.json und den Rohjournalen enthalten. Ein CSV-Datensatz enthält einen XYZ-Messpunkt, keine drei getrennten Zeitpunkte.

Die monotone Host-Uhr dokumentiert den Abschluss der Messpunktlesung, der Fensterbildung und der Entscheidung. UTC dient der Zuordnung von Bedienereignissen. Nominell dauert ein nicht überlappendes Fenster $128/200 = 0,640$ s. Aus den Host-Zeitstempeln ergeben sich in den ausgewählten Aufnahmen rund 207 XYZ-Punkte pro Sekunde beziehungsweise etwa $0,617$ s pro Fenster. Diese Beobachtung ist keine unabhängige Messung des internen Sensortakts; die Abweichung bleibt offengelegt. Die Fensterfüllzeit gehört zur praktischen Alarmverzögerung, nicht zur nachfolgend gemessenen reinen Verarbeitung eines bereits vollständigen Fensters.

Bei erkannten Datenlücken, Überläufen oder Sättigung wird ein Fenster nicht stillschweigend als normal gewertet. Unvollständige Reste werden gespeichert, aber nicht durch künstliche Werte zu vollständigen Fenstern ergänzt. Die begrenzte Queue umfasst vier Fenster. Im gemeinsamen Lauf wurde höchstens ein wartendes Fenster protokolliert.

6.4 Vorverarbeitung und Modellstände

Der aktuelle Pfad verarbeitet die Achsen X, Y und Z in fester Reihenfolge als Float32. Er verwendet die auf normalen Trainingsdaten bestimmten Mittelwerte μ und Standardabweichungen σ : $z = (x - \mu) / \sigma$. Der gleiche Scaler wird für alle drei Methoden eingesetzt. Im aktuellen Handy-Pfad wird vorher kein eigener Mittelwert je Fenster entfernt. Das ist ein wesentlicher Unterschied zum früheren v4-Pfad mit fensterweiser Zentrierung. Die beiden Modellstände dürfen deshalb trotz gleicher Hardware und gleicher Fenstergröße nicht vermischt werden.

Tabelle 6-2: Festgelegte Skalierung des aktuellen Normalprofils, gerundet

Achse	Trainingsmittelwert μ [g]	Trainingsstandardabweichung σ [g]
X	-0,624318715	0,028777793
Y	-0,710917289	0,024151856
Z	0,096116970	0,080470853

Der RMS-Score ist die Quadratwurzel des Mittelwerts aller 384 quadrierten standardisierten Werte eines Fensters. Er ist ein dimensionsloser aggregierter Kennwert und nicht der unskalierte Beschleunigungs-RMS in g. Isolation Forest verarbeitet dieselben 384 Werte als abgeflachten Merkmalsvektor. Verwendet werden 200 Bäume, random_state = 42, max_samples = auto, contamination = auto und n_jobs = 1. Der Anomaliescore ist -score_samples; die Entscheidung folgt der separat eingefrorenen P99-Schwelle und nicht unmittelbar der eingebauten predict-Grenze von scikit-learn.

Der Autoencoder rekonstruiert das standardisierte 128×3 -Fenster. Sein Score ist der mittlere quadratische Rekonstruktionsfehler über alle Zeitpunkte und Achsen. Die Architektur besitzt 507 trainierbare Parameter. Ihre Topologie bleibt gegenüber dem

früheren Bericht gleich; gleichbleibende Topologie bedeutet jedoch nicht identische Gewichte oder identische Kalibrierung.

Tabelle 6-3: Architektur des verwendeten Autoencoders

Stufe	Operation	Ausgabeform ohne Batchdimension
Eingabe	Standardisierte XYZ-Werte	128×3
Encoder 1	Conv1D, 8 Filter, Kernel 5; MaxPooling 2	64×8
Encoder 2	Conv1D, 4 Filter, Kernel 3; MaxPooling 2	32×4
Decoder 1	Conv1D, 4 Filter, Kernel 3; Upsampling 2	64×4
Decoder 2	Conv1D, 8 Filter, Kernel 3; Upsampling 2	128×8
Rekonstruktion	Conv1D, 3 Filter, Kernel 5; linear	128×3

6.5 Normaltraining, Schwellen und Exportprüfung

Das Profil wurde vor den Handyversuchen aus vier normalen Aufnahmen von jeweils nominell 60 s bei 75 % PWM erstellt. Die ersten drei vollständigen Dateien dienen dem Training, die vierte der Validierung. Es entstehen 291 Trainings- und 97 Validierungsfenster. Der Scaler wird ausschließlich anhand der 37.248 XYZ-Punkte der Trainingsfenster angepasst. Die Aufteilung erfolgt nach vollständigen Aufnahmen; benachbarte Fenster derselben Datei werden nicht zufällig auf Training und Validierung verteilt.

Der Autoencoder wurde mit Batchgröße 32 und maximal 100 Epochen trainiert. Die dokumentierte beste Epoche ist 100; ein früherer Abbruch wurde nicht ausgelöst. Die abschließenden Verluste betragen etwa 0,25145 für Training und 0,27893 für Validierung. Diese Verluste sind Normaldaten-Rekonstruktionsfehler und keine Erkennungsquoten für die spätere Handyvibration.

Für jede Methode wird der Schwellenwert als 99. Perzentil der Scores der separaten normalen Validierungsaufnahme bestimmt. Verwendet wird lineare Quantilinterpolation. Bei 97 sortierten Scores entspricht die Nullbasisposition $(97 - 1) \cdot 0,99 = 95,04$ einer Interpolation zwischen dem 96. und 97. geordneten Wert. Ein Alarm gilt bei Score > Schwelle. Die gleiche Perzentilregel bedeutet weder gleiche numerische Scorebereiche noch garantiert gleiche Fehlalarmraten auf neuen Daten.

Tabelle 6-4: Eingefrorene Schwellen der Handyreihe

Methode	Score	Schwelle
Autoencoder	MSE der standardisierten Rekonstruktion	0,477202599144
Isolation Forest	-score_samples	0,483354740160
RMS	RMS der standardisierten Werte	1,296361072562

Die Autoencoder-Schwelle stammt aus der eingefrorenen Keras-Validierung. Der Float32-TFLite-Export wird auf denselben 97 Validierungsfenstern geprüft: Die größte absolute

Rekonstruktionsabweichung beträgt $2,623 \cdot 10^{-6}$, die größte MSE-Abweichung $7,192 \cdot 10^{-8}$; keine Entscheidung ändert sich. Ein Validierungsfenster liegt oberhalb der P99-Grenze. Dieser Kalibrierungsbefund ist keine unabhängig gemessene Fehlalarmrate im späteren Betrieb. Modelle und Schwellen werden nach Sichtung der Handyversuche nicht optimiert.

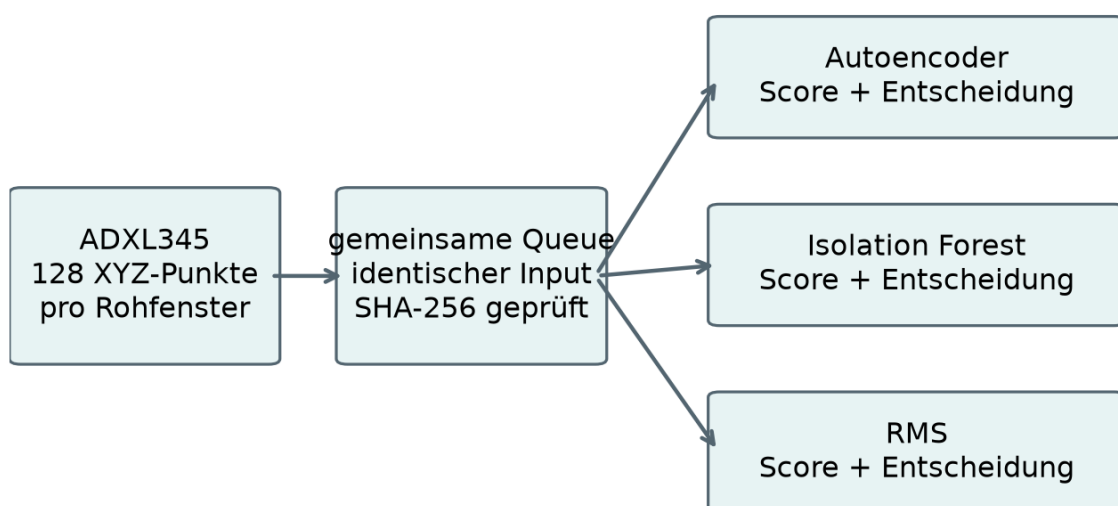
6.6 Einzelbetrieb, Replay und gemeinsame Live-Auswertung

Im Einzelbetrieb läuft jeweils nur eine Methode im Aufnahmeprozess. Erfassung, Qualitätsprüfung, Vorverarbeitung, Entscheidung und Protokollierung gehören dennoch zum Prozess. Vor Beginn werden 20 Aufwärmfenster verarbeitet. Es läuft keine Live-GUI. Die drei ausgewählten Einzelaufnahmen enthalten unterschiedliche physikalische Anregungen; ein identisches Handymodell allein macht verschiedene Anrufe nicht zu identischen Sensoreingaben.

Beim Offline-Replay werden alle drei Methoden auf die gleichen gespeicherten Rohfenster jeder Einzelaufnahme angewendet. Dadurch ist der Eingabevergleich exakt reproduzierbar. Die Reihenfolge der ersten Alarmfenster lässt sich vergleichen, die Replay-Laufzeit ist jedoch keine gemessene Live-Reaktion auf eine neue Handyvibration. Die erneute Auswertung erzeugt keine zusätzlichen unabhängigen physikalischen Ereignisse.

Im gemeinsamen Live-Lauf erfasst ein Sensorpfad die Rohdaten. Jedes vollständige Fenster wird an drei Worker-Threads im selben Prozess übergeben. Ein gemeinsamer Startmechanismus koordiniert die Auswertung; gleiche Startbedingungen garantieren keine identische CPU-Zuteilung. Für jedes Fenster werden Eingabehash, Dispatch-, Worker-Start- und Entscheidungszeit gespeichert. Die feste Reihenfolge der Zeilen im Journal ist nicht das Ergebnisranking. Dieses wird aus den gemessenen Entscheidungszeitpunkten berechnet.

Gemeinsamer Live-Lauf: ein Prozess, drei Worker-Threads



Einzelbetrieb: jeweils nur ein Verfahren. Replay: gespeicherte Rohfenster erneut auswerten.

Abbildung 6-2: Datenfluss des gemeinsamen Live-Vergleichs. Alle drei Methoden erhalten dasselbe Rohfenster; ihre Verarbeitung und Ausgabezeitpunkte bleiben getrennt protokolliert. Quelle: eigene Darstellung anhand der Implementierung und Journale in B12.

Tabelle 6-5: Zeitbegriffe und zulässige Interpretation

Größe	Bezugspunkte	Bedeutung
Verarbeitungszeit	Start bis Ende des jeweiligen Auswerteraufrufs	Vorverarbeitung, Modellbewertung und Score im instrumentierten Aufruf; kein einheitlicher reiner Modellkern
Fenster-bis-Entscheidung	Host-Abschluss des letzten Fensterpunkts bis Entscheidung	Einschließlich Übergabe und Scheduling; ohne vorherige Fensterfüllzeit
Erstes Alarmfenster	Erster Fensterindex mit Score > Schwelle	Vergleich der Eingabesensitivität bei identischen Fenstern
Relative Alarmausgabe	Erste Alarmausgabe einer Methode minus früheste Alarmausgabe	Beobachteter Ausgabevorsprung im selben Live-Lauf
Physische Erkennungsverzögerung	Entscheidung minus unabhängig gemessener Vibrationsbeginn	Mit den vorliegenden Bedienmarkern nicht bestimmbar

6.7 Ablauf, Bedienmarker und Annahmen

Das vorbereitete Vorgehen sieht mindestens 30 s Ruhe vor der Anruaufforderung und mindestens 15 s Nachlauf nach der gemeldeten Beendigung vor. Der Assistent übernimmt Start, Protokollierung, Auswertung und Diagrammerstellung; die Person am Aufbau löst den Anruf aus. Ein erster Anlauf wird bei unklarer Abstimmung nicht rückwirkend als sauber synchronisiert bezeichnet, sondern erhalten und getrennt dokumentiert.

Der Marker `call_requested` bezeichnet die protokollierte Aufforderung, nicht den tatsächlichen Beginn der Handyvibration. Ebenso ist die Rückmeldung zum Ende keine automatische Messung des mechanischen Abklingens. Die geplanten ungefähr 5 s Vibration werden nicht als tatsächlich gemessene Dauer übernommen. Der Zeitraum zwischen den beiden Bedienmarkern umfasst auch Reaktions-, Verbindungs- und Rückmeldezeit. Die Abbildungen schattieren diesen Bedienzeitraum ausdrücklich, nicht ein verifiziertes Störungsintervall.

Fenster werden anhand ihrer Anfangs- und Endzeit in vor der Aufforderung, zwischen den Markern, nach der Rückmeldung und markerübergreifend eingeteilt. Markerübergreifende Fenster bleiben gesondert erhalten. Ohne unabhängige physische Referenz ist die Anzahl markierter Fenster eine deskriptive Alarmzahl. Elf Alarmfenster bedeuten weder elf unabhängige Störungen noch automatisch eine bessere Erkennungsquote als sechs Alarmfenster.

Tabelle 6-6: Annahmen und offene Nachweise

Annahme / Größe	Vorliegender Nachweis	Konsequenz
Gleiche Eingaben	Gemeinsamer Rohstrom und Hash je Fenster im Live-Vergleich; identische Rohdateien im Replay	Direkter Methodenvergleich innerhalb dieser Modi möglich
Gleiche Anrufe im Einzelbetrieb	Keine unabhängige Gleichheitsmessung	Keine direkte Rangfolge aus getrennten Startzeiten
Übertragbarkeit des Normalprofils	Vorheriges Normaltraining; endgültige Handygeometrie nicht vollständig bestätigt	Ergebnisse bleiben auf den Pilotaufbau begrenzt
Physischer Beginn / Ende	Nur Bedienmarker und Nutzerrückmeldung	Keine absolute Fehler-Erkennungsverzögerung oder bestätigte F1-Auswertung
Elektrischer Verbrauch	Keine Spannungs-/Strommessung	CPU und RAM werden nicht in Watt oder Wh umgerechnet
Wiederholbarkeit	Ein ausgewählter Anruf je Einzelmethode, ein gemeinsamer Lauf	Keine Signifikanz- oder allgemeine Zuverlässigkeitsaussage

6.8 Dateien, Visualisierung und Nachvollziehbarkeit

Die Recorder speichern raw.csv, decisions.csv, run.json und cues.jsonl. Im gemeinsamen Lauf kommt resources.csv hinzu. Quelle, Modellpaket, Laufzustand, Sensor-/PWM-Rücklesungen und Implementierungskennung sind den jeweiligen Laufdateien zugeordnet. Die aktuelle Auswertung stützt sich auf die abgeschlossenen Laufjournale und Ergebnisberichte. Der erhaltene Vorbereitungstext protocol.md ist ein Planungsstand und darf mit seinem damaligen Status nicht als abschließender Ergebnisbericht gelesen werden.

Die vorhandene GUI ermöglicht die Darstellung der drei Verfahren und ihrer Schwellen. Für diese Versuche wurde jedoch bewusst ohne Live-GUI aufgenommen; GUI-Zusatzlast wurde somit nicht gemessen. Diagramme werden nach Abschluss aus gespeicherten Daten erzeugt. Der vorhandene Screenshot des Ergebnisberichts dokumentiert dessen Darstellung, nicht eine während der Aufnahme beobachtete GUI. Die Präsentation unter B12 und die Word-Abbildungen unter B13 beruhen auf denselben Messwerten; die Word-Diagramme wurden für lesbare Beschriftungen neu gesetzt.

7 Evaluation

7.1 Umfang und Auswahl der Aufnahmen

Die Handyreihe vom 19. September enthält sechs Einzelanläufe und einen gemeinsamen Live-Lauf. Drei Einzelanläufe wurden nach Abstimmungsproblemen durch spätere Aufnahmen ersetzt. Sie bleiben vollständig im Archiv und werden nicht als fehlende oder gelöschte Daten behandelt. Die Auswahl folgt der protokollierten Anrufkoordination, nicht einer nachträglichen Auswahl der günstigsten Modelleistung. Auch die ausgewählten Läufe besitzen nur Bedienmarker und keine verifizierte physikalische Ereigniszeit.

Tabelle 7-1: Erhaltene Anläufe und Auswertungsrolle

Kurzname	Vollständige Fenster	Alarmfenster	Rolle
AE01	170	30	Früher Anlauf; möglicher zu früher Anruf, wiederholt
AE02	158	12	Erneuter Abstimmungsversuch; ein ungültiges Fenster, wiederholt
AE03	184	11	Ausgewählter Autoencoder-Einzelprozess
IF01	154	7	Anrufkoordination unklar/verpasst, wiederholt
IF02	196	6	Ausgewählter Isolation-Forest-Einzelprozess
RMS01	234	6	Ausgewählter RMS-Einzelprozess
Gemeinsam01	197 je Methode	AE 11 / IF 7 / RMS 6	Gemeinsamer Live-Vergleich

Technische Funktionsprüfungen im Ordner `technical_checks` sind nicht Teil dieser Ergebnisstichprobe. AE03, IF02 und RMS01 entsprechen den vollständigen Ordernamen `autoencoder_awaiting_call_03`, `isolation_forest_awaiting_call_02` und `rms_awaiting_call_01`. Gemeinsam01 liegt unter `joint_trials/all_methods_01`. Diese Zuordnung verhindert die Verwechslung von Wiederholungsanläufen und ausgewerteten Aufnahmen.

7.2 Datenintegrität und Reproduzierbarkeit

Die drei ausgewählten Einzelaufnahmen enthalten zusammen 78.843 XYZ-Punkte, 614 vollständige Fenster und 251 gespeicherte Restpunkte. Der gemeinsame Lauf enthält 25.270 XYZ-Punkte, 197 vollständige Fenster und 54 Restpunkte. Damit liegen 811 verschiedene vollständige Rohfenster der vier ausgewählten Aufnahmen vor. Der gemeinsame Lauf erzeugt daraus 591 Entscheidungen; zusammen mit den 614 Einzelentscheidungen sind es 1.205 ursprüngliche Live-Entscheidungen. Diese Anzahl ist keine Zahl unabhängiger Versuche.

In diesen vier Aufnahmen wurden keine ungültigen Modellfenster, verworfenen Fenster, Gap- oder Overrun-Ereignisse und keine Sättigungsflags protokolliert. Im gemeinsamen Lauf verblieben nach Abschluss keine vollständigen unverarbeiteten Fenster. Solche Nullbefunde

belegen die protokollierte Integrität, nicht die Abwesenheit aller denkbaren unbekannten Messfehler.

Alle 614 ursprünglichen Einzelentscheidungen lassen sich beim Replay exakt reproduzieren; die größte Scoreabweichung beträgt 0. Auch die 591 Entscheidungen des gemeinsamen Laufs wurden aus denselben Rohfenstern exakt reproduziert. Für alle 197 gemeinsamen Fenster stimmen die Eingabehashes der drei Methoden überein. Zusätzlich wurden vor dem gemeinsamen Lauf 1.842 Auswertungen der drei Methoden auf den 614 gespeicherten Einzelfenstern abgeglichen. Diese Softwareprüfungen bestätigen Konsistenz, nicht zusätzliche unabhängige Erkennungsereignisse.

Tabelle 7-2: Ausgewählte Aufnahmen und Verteilung der gültigen Fenster

Aufnahme	Rohpunkte / Rest	Vor Marker	Zwischen Markern	Nach Marker / übergreifend
AE03	23.617 / 65	83	52	47 / 2
IF02	25.170 / 82	76	59	59 / 2
RMS01	30.056 / 104	83	109	40 / 2
Gemeinsam01	25.270 / 54	77	79	39 / 2

7.3 Einzelbetrieb und Vergleich auf identischen Replay-Fenstern

Im ausgewählten Einzelbetrieb markiert der Autoencoder elf Fenster, Isolation Forest sechs und RMS sechs. Die Alarme liegen jeweils vollständig zwischen den Bedienmarkern. Vor der Anruaufforderung und nach der Enderückmeldung treten in diesen ausgewählten Läufen keine Alarme auf. Weil Anrufe und Zeitverläufe voneinander abweichen, sind diese Zahlen allein weder ein Geschwindigkeitsranking noch eine faire Erkennungsquote.

Der Offline-Vergleich wertet deshalb jede dieser drei Rohaufnahmen mit allen drei eingefrorenen Methoden aus. Auf AE03 markieren alle Methoden erstmals Fenster 97. Auf IF02 markiert der Autoencoder Fenster 93, die beiden Baselines erst Fenster 94. Auf RMS01 beginnen alle drei bei Fenster 155. Der Autoencoder ist somit in diesen drei Aufnahmen nie später im ersten Alarmfenster, einmal früher und zweimal gleichauf. Die drei Befunde stammen von drei verschiedenen Anrufen; zusätzliche Verallgemeinerung erfordert mehr unabhängige Wiederholungen.

Tabelle 7-3: Replay derselben Rohfenster; erstes Alarmfenster und gesamte Alarmfenster

Aufnahme	Autoencoder	Isolation Forest	RMS
AE03	erstes 97; 11 Alarme	erstes 97; 6 Alarme	erstes 97; 8 Alarme
IF02	erstes 93; 9 Alarme	erstes 94; 6 Alarme	erstes 94; 6 Alarme
RMS01	erstes 155; 9 Alarme	erstes 155; 6 Alarme	erstes 155; 6 Alarme

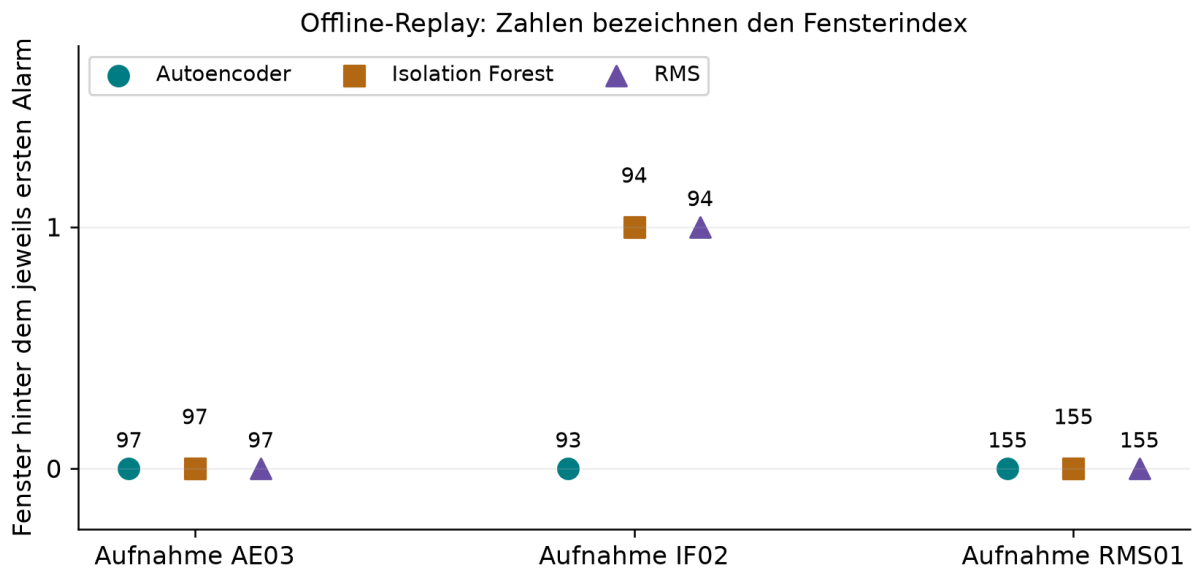


Abbildung 7-1: Erstes Alarmfenster im Offline-Replay. Die Zahlen an den Punkten sind Fensterindizes; die vertikale Achse zeigt den Abstand zum frühesten Alarmfenster derselben Aufnahme. Eine Null bedeutet Gleichstand auf Fensterebene, keine gleichzeitig gemessene Live-Ausgabe. Quelle: B12, replay_v1/summary.json.

7.4 Gemeinsamer Live-Lauf: Signal und Alarmreihenfolge

Im gemeinsamen Lauf erhalten alle Methoden dieselben 197 Fenster. Die Anrufofforderung liegt 48,019 s, die protokollierte Enderückmeldung 97,322 s nach dem Aufnahmestart. Dazwischen befinden sich 79 vollständige Fenster. Der Autoencoder markiert elf, Isolation Forest sieben und RMS sechs davon. In den 77 Fenstern vor der Aufforderung, den 39 Fenstern nach der Rückmeldung und den zwei markerübergreifenden Fenstern gibt es bei keiner Methode einen Alarm. Diese Nullwerte beschreiben die gespeicherte Folge, nicht eine nachgewiesene allgemeine Fehlalarmrate von 0 %.

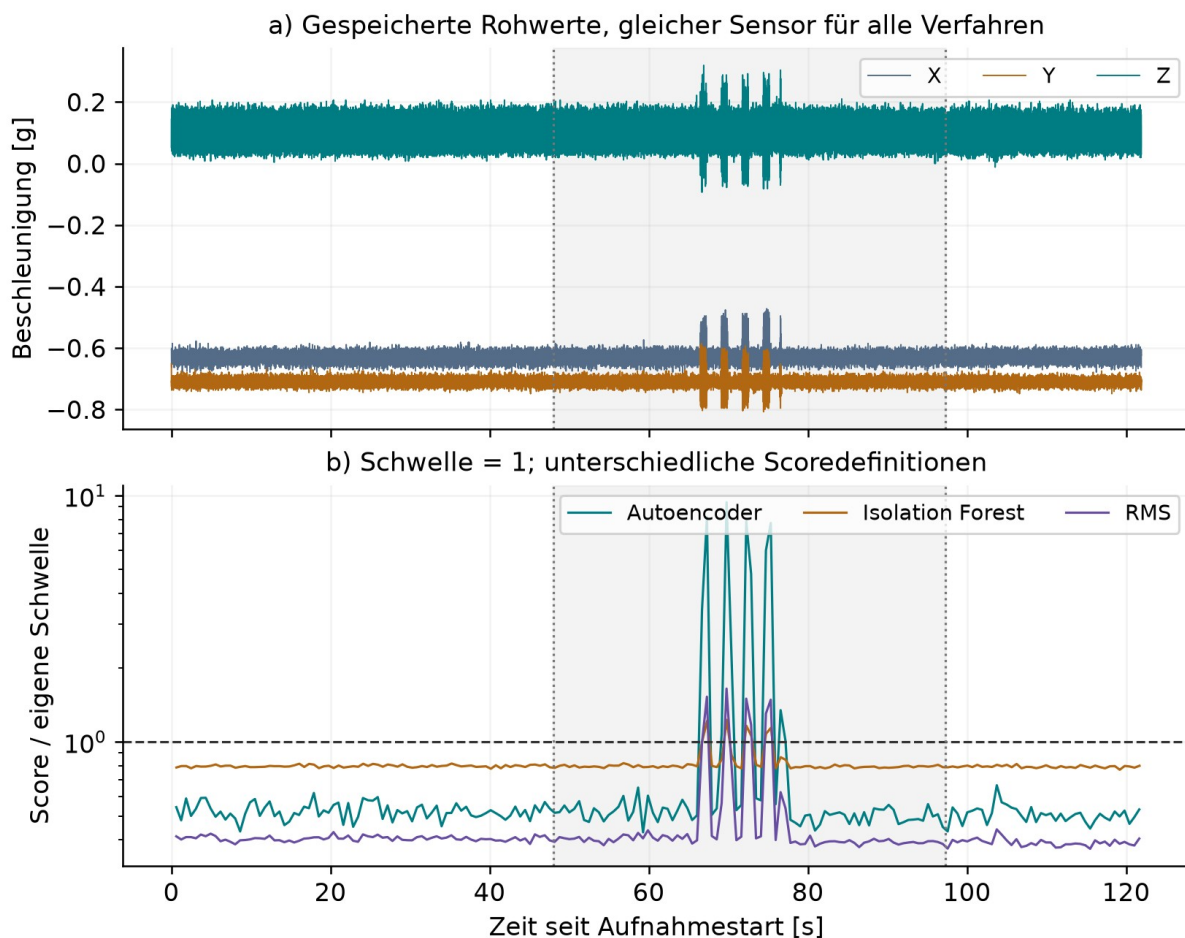


Abbildung 7-2: Gemeinsamer Rohdaten- und Scoreverlauf. Graue Bereiche liegen zwischen den Bedienmarkern, nicht zwischen unabhängig gemessenem Vibrationsbeginn und -ende. Die Scoreachse ist logarithmisch und auf die jeweilige Schwelle normiert; der Wert 1 kennzeichnet die Alarmgrenze. Quelle: B12, Gemeinsam01.

Tabelle 7-4: Erste Alarme im gemeinsamen Live-Lauf

Methode	Erstes Alarmfenster	Rang des Fensters	Rang der Ausgabe	Abstand zur ersten Ausgabe [ms]
Autoencoder	108	1, gleichauf mit IF	1	0,000
Isolation Forest	108	1, gleichauf mit AE	2	19,061
RMS	109	2	3	615,659

Der Autoencoder und Isolation Forest überschreiten ihre Schwelle erstmals auf demselben Fenster 108. Die Autoencoder-Entscheidung liegt um 19,061 ms früher vor. RMS benötigt wesentlich weniger Rechenzeit, markiert dieses Fenster aber noch nicht als auffällig. Seine erste Alarmausgabe erfolgt für Fenster 109 und damit 615,659 ms nach derjenigen des Autoencoders. Die Trennung zwischen erstem Alarmfenster und Ausgabezeitpunkt ist entscheidend: Das spätere RMS-Ergebnis entsteht hier nicht durch langsame RMS-Berechnung, sondern überwiegend durch die spätere Schwellenüberschreitung.

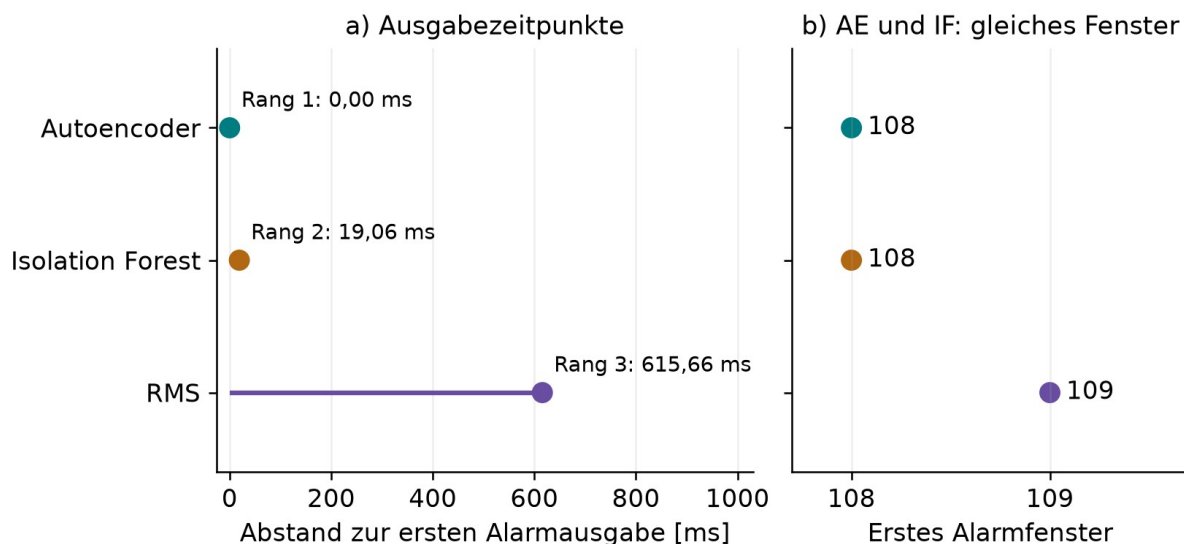


Abbildung 7-3: Nummerierte Reihenfolge der ersten Alarmausgaben im gemeinsamen Lauf und zugehörige Fensterindizes. Null ist der Zeitpunkt des ersten Autoencoder-Alarms, nicht der physische Störungsbeginn. AE und IF sind auf Fensterebene gleichauf; die Ausgabezeitpunkte unterscheiden sich. Quelle: B12, Entscheidungen von Gemeinsam01.

Die relative Zeitdifferenz wird aus der monotonen Uhr berechnet: $\Delta t_m = (t_{m,erste\ Alarmausgabe} - \min_k t_{k,erste\ Alarmausgabe}) / 10^6$ in Millisekunden. Dabei werden die tatsächlichen Entscheidungszeitpunkte verwendet, nicht die Reihenfolge der Methodenliste oder der CSV-Zeilen. Die Daten belegen den Ausgabevorsprung des Autoencoders für diesen Lauf. Eine absolute Reaktionszeit ab Beginn der Handyvibration ist mangels unabhängiger Referenz nicht bestimmbar.

7.5 Warum unterscheiden sich die ersten Alarme?

Tabelle 7-5: Unmittelbarer Schwellenvergleich auf dem gemeinsamen Fenster 108

Methode	Score	Eingefrorene Schwelle	Entscheidung
Autoencoder	1,631938754	0,477202599	Alarm
Isolation Forest	0,484321113	0,483354740	Alarm
RMS	1,282930084	1,296361073	Normal

Auf Fenster 108 liegt der Autoencoder-Fehler deutlich oberhalb seiner Normalgrenze. Isolation Forest liegt nur knapp darüber. RMS liegt knapp darunter und überschreitet seine Schwelle erst in Fenster 109 mit einem Score von etwa 1,97525. Fenster 107 ist noch bei allen drei Methoden unauffällig. Die Werte erklären die protokollierte Rangfolge direkt und ohne eine nachträgliche Anpassung der Schwellen.

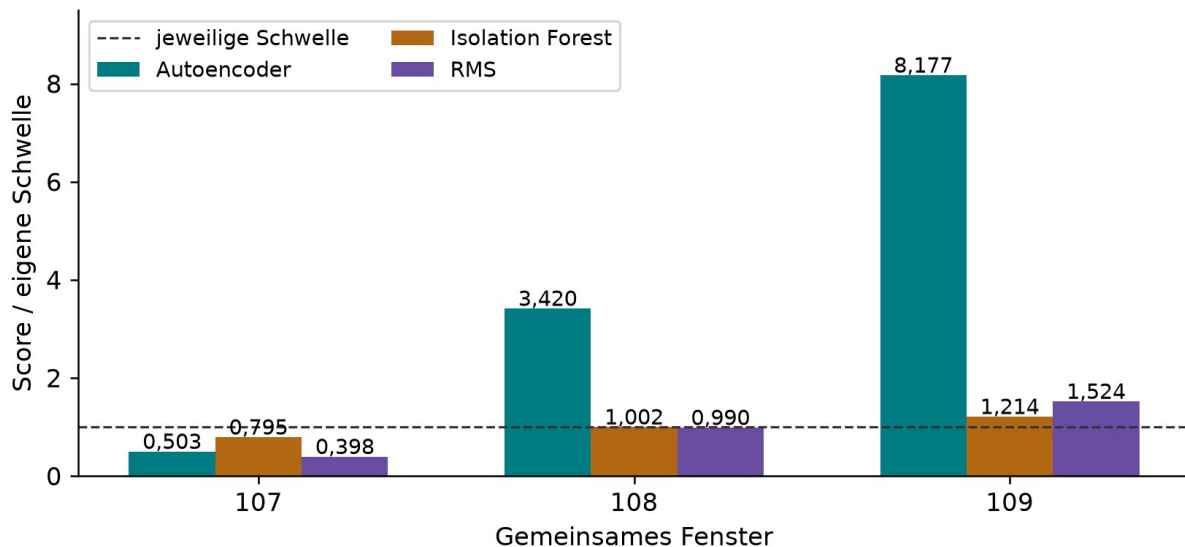


Abbildung 7-4: Schwellenübergang der Fenster 107 bis 109. Die Normierung erleichtert das Ablesen der jeweiligen Grenzüberschreitung; die Quotienten sind weder Wahrscheinlichkeiten noch untereinander kalibrierte Störungsschweregrade. Quelle: B12, Gemeinsam01.

Als methodische Erklärung ist plausibel, dass der Autoencoder Veränderungen der gelernten Rekonstruktionsstruktur erfasst, während RMS die standardisierten Werte auf einen einzigen Energiekennwert verdichtet. Der Isolation Forest bewertet die Lage des gesamten abgeflachten Fensters im gelernten Merkmalsraum. Diese unterschiedlichen Scorefunktionen, ihre Normaldatenkalibrierung und die konkrete Lage der Störung im Fenster beeinflussen die erste Überschreitung. Eine isolierte kausale Überlegenheit der Architektur lässt sich aus einem Ereignis nicht ableiten. Dafür müssten unter anderem Normalprofil, Vorverarbeitung und Schwellencharakteristik gezielt variiert werden.

Die Verarbeitung des ersten Alarmfensters dauert beim Autoencoder etwa 2,789 ms und beim Isolation Forest etwa 21,733 ms. Der Ausgabeabstand von 19,061 ms entsteht im selben gemeinsam gestarteten Fenster und ist mit diesem Aufwandsunterschied vereinbar; Scheduling und weitere Übergaben sind ebenfalls enthalten. RMS benötigt im ersten eigenen Alarmfenster nur etwa 0,183 ms. Rechenzeit und frühzeitige Anomalieausgabe sind somit unterschiedliche Bewertungskriterien.

7.6 Rechenaufwand und Verhalten des Raspberry Pi

Die Hardware wird für die neue Reihe nicht geändert. Neu gemessen werden jedoch Prozessauslastung und zeitliches Verhalten im tatsächlich verwendeten Recorder. Die Einzelprozesse und der gemeinsame Prozess besitzen unterschiedliche aktive Methoden, Bibliotheken und Konkurrenzsituationen. Identische Hardware begründet deshalb keine identischen CPU-, RAM- oder Latenzwerte. Die bisherigen Ressourcennmessungen werden im Anhang unverändert erhalten und nicht durch die neuen Zahlen ersetzt.

Tabelle 7-6: Verarbeitungszeit im jeweiligen Auswerteraufruf; ms

Methode	Einzelbetrieb Median	Einzelbetrieb P95	Gemeinsam Median	Gemeinsam P95
Autoencoder	0,459	0,585	2,040	5,571
Isolation Forest	23,429	32,061	23,237	31,527
RMS	0,285	0,352	0,264	0,317

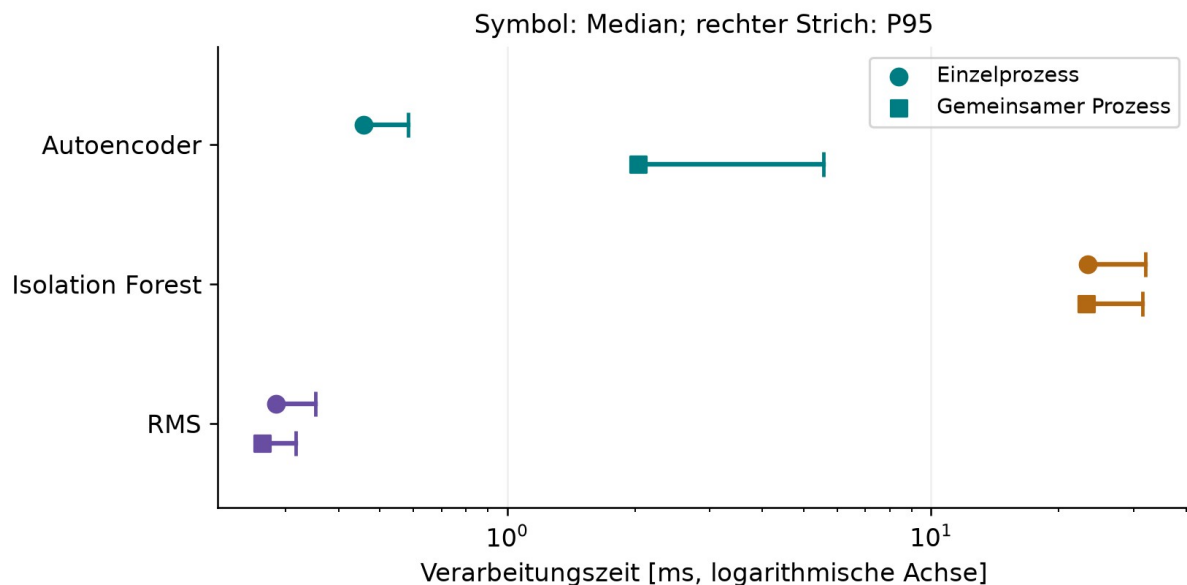


Abbildung 7-5: Verarbeitungszeiten im Einzelbetrieb und im gemeinsamen Prozess. Symbole zeigen Median, rechte Striche P95; die Zeitachse ist logarithmisch. Erfasst wird der instrumentierte Auswerteraufwurf einschließlich seiner Vorverarbeitung, nicht eine einheitlich isolierte Modellkernzeit. Quelle: B12, ausgewählte decisions.csv.

Der Isolation Forest benötigt in beiden Modi die meiste Verarbeitungszeit. RMS ist am schnellsten berechnet. Der Autoencoder liegt dazwischen; seine gemessene Zeit steigt im gemeinsamen Prozess. Diese Beobachtung ist mit zusätzlicher Konkurrenz und anderem Scheduling vereinbar, ist aber ohne kontrollierte Wiederholungsmatrix keine isolierte Ursache-Wirkungs-Messung. Die Einzelaufnahmen unterscheiden sich außerdem in Dauer und Ereignisverlauf.

Die CPU-Kennzahl ist der Median der im jeweiligen Lauf protokollierten Prozessmessungen; 100 % entsprechen einem logischen Kern. Sie umfasst den gesamten Aufnahmeprozess einschließlich Erfassung und Protokollierung. Der RSS-Wert ist der größte während der protokollierten Fenster abgetastete residente Prozessspeicher. Er ist weder der gesamte RAM des Raspberry Pi noch zwingend der höchste Wert während Import, Modellstart oder Prozesslebensdauer. Ein Punkt zwischen zwei Messproben kann unbemerkt bleiben.

Tabelle 7-7: Prozessressourcen der ausgewählten Handyaufnahmen

Betriebsart	CPU-Median [% eines Kerns]	Größter abgetasteter RSS [MiB]	Umfang
Autoencoder allein	9,72	48,80	184 Fenster
Isolation Forest allein	12,98	167,11	196 Fenster
RMS allein	9,69	39,31	234 Fenster
Alle drei gemeinsam	13,04	174,42	197 gemeinsame Fenster

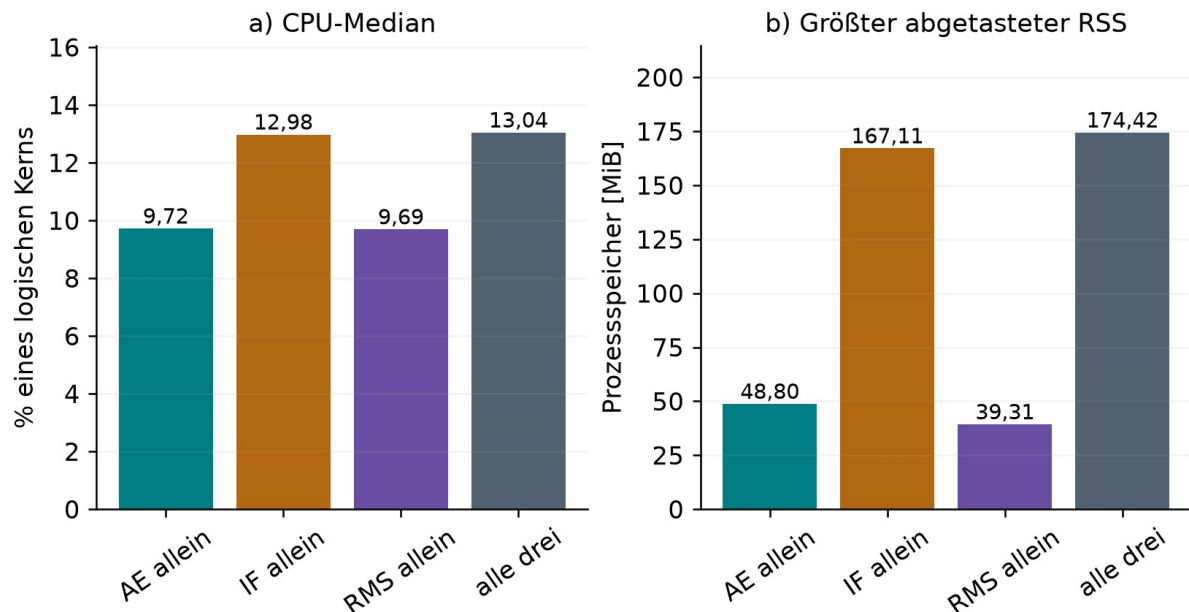


Abbildung 7-6: CPU und RAM im Einzelbetrieb und gemeinsamen Live-Prozess. Es handelt sich um Prozessmessungen mit unterschiedlichen Laufdauern; die gemeinsame Auslastung darf nicht pro Methode aufgeteilt oder aus den Einzelwerten addiert werden. Quelle: B12, decisions.csv und resources.csv.

Die gemeinsame Speichernutzung liegt in diesem Lauf etwas oberhalb des Isolation-Forest-Einzelprozesses, nicht bei der Summe aller Einzelprozesse. Im gemeinsamen Prozess werden Teile der Erfassung, Laufzeitumgebung und geladenen Bibliotheken geteilt. Eine genaue Zuordnung einzelner Speicheranteile wurde nicht instrumentiert. Die Ergebnisse zeigen im untersuchten Umfang eine Ausführung ohne protokollierten Fensterrückstand oder Datenverlust; sie sind kein unbegrenzter Dauerbetriebsnachweis.

Im gemeinsamen Lauf liegt das empirische P99 der Zeit vom vollständigen Fenster bis zur Entscheidung bei rund 9,156 ms für AE, 45,008 ms für IF und 8,793 ms für RMS. Die größten beobachteten Werte betragen 12,205 ms, 83,893 ms und 11,455 ms. Sie liegen deutlich unter dem nominellen 640-ms-Fensterbudget. Dieses Ergebnis ist von der früheren vorab definierten Laufzeitmatrix zu unterscheiden; der damalige vollständige Fristtest bleibt separat im Anhang dokumentiert. Ein kleines Verarbeitungs-P99 beseitigt weder die Fensterfüllzeit noch die Unsicherheit des physikalischen Störungsbeginns.

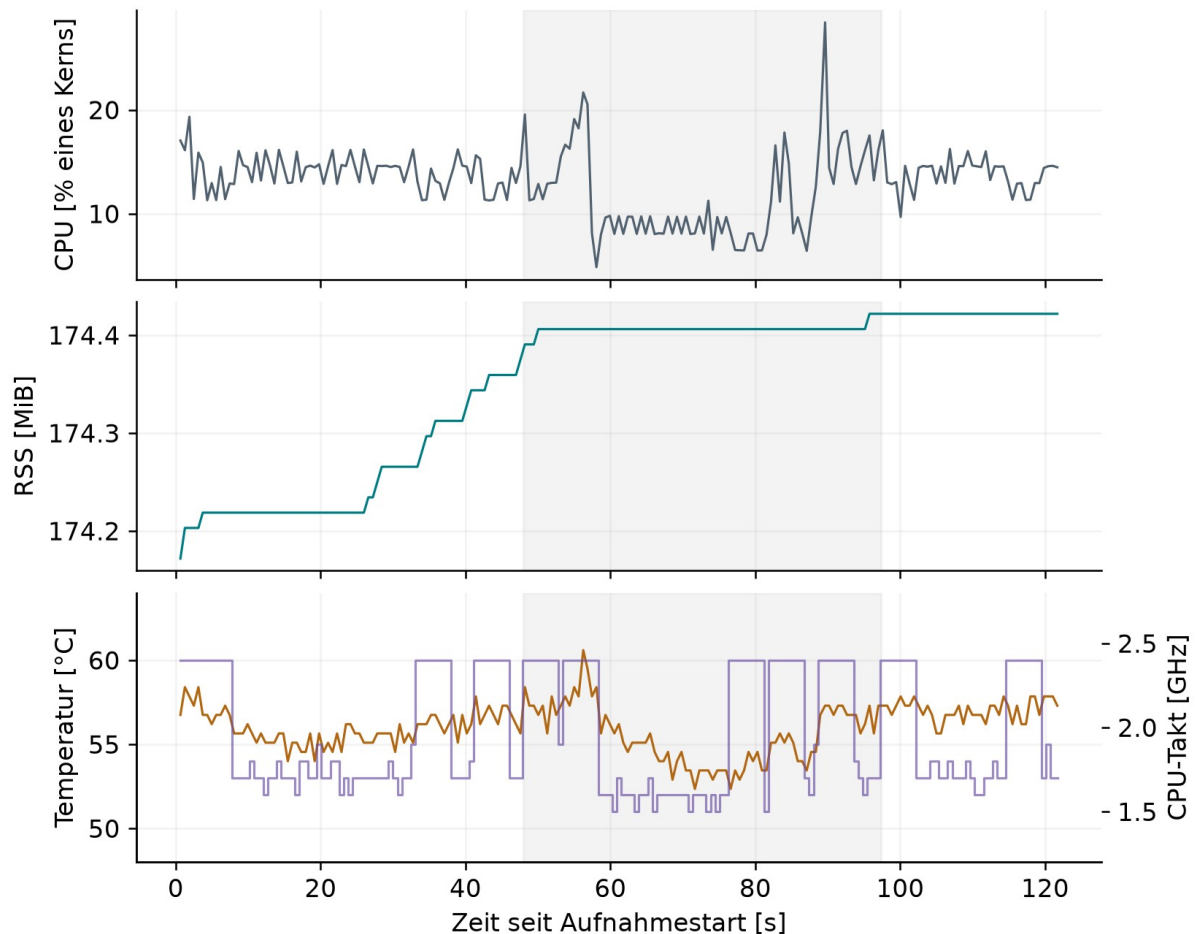


Abbildung 7-7: Zeitlicher Ressourcenverlauf des gemeinsamen Prozesses. Grau: Zeitraum zwischen Bedienmarkern. Temperatur und CPU-Takt sind zusätzliche Betriebsdiagnosen, keine Leistungs- oder Energiemessung. Quelle: B12, Gemeinsam01/resources.csv.

Im gemeinsamen Lauf wurden Temperaturen von 52,35 bis 60,60 °C und CPU-Frequenzen von 1,5 bis 2,4 GHz protokolliert. Über die ausgewählten Einzelprozesse reichen die Temperaturproben von 51,80 bis 61,15 °C. Aus wechselnder CPU-Frequenz allein folgt kein Nachweis thermischer Drosselung. Für diese Reihe liegt keine vollständige unabhängige Drosselungsanalyse vor. Ein unauffälliger kurzer Versuch schließt spätere Temperatur- oder Speicherprobleme nicht aus.

Unter Verbrauch wird in dieser Auswertung ausschließlich der beobachtete Rechen- und Speicherbedarf verstanden. Elektrische Leistung in Watt, Stromaufnahme und Energie in Wh wurden nicht gemessen. CPU-Auslastung ist kein hinreichender Ersatz dafür. Auch die externe 12-V-Lüfterversorgung gehört nicht automatisch zum erfassten Raspberry-Pi-Prozessbedarf. Eine Aussage wie „der Autoencoder verbraucht weniger elektrische Energie“ wird deshalb nicht aus diesen Daten abgeleitet.

7.7 Einordnung der früheren Prüfreihe

Die frühere Untersuchung vom 11./12. September verwendete dasselbe Hardwareprinzip, aber das v4-Paket mit fensterweiser Zentrierung, anderen Trainingsaufnahmen, Gewichten, Skalierungen und Schwellen. Die Anregung war eine separat befestigte Platte zur Änderung

des Luftstroms. Die Handyreihe untersucht eine andere äußere Einwirkung. Ein Unterschied der Ergebnisse ist deshalb nicht allein einer geänderten Anregung oder allein einer Methode zurechenbar.

Die negativen früheren Ergebnisse bleiben erhalten: In der vollständigen Folge Normal → Luftstromänderung → Normal markierten RMS sieben, IF vier und AE keines von 194 gültigen Fenstern des geänderten Zustands. Die F1-Werte dieser gesamten historischen Folge betrugen 6,93 %, 4,04 % und 0 %. H1 war dort nicht bestätigt. Die neue frühere Autoencoder-Alarmausgabe macht aus diesem alten Ergebnis keinen höheren F1-Wert. Tabellen A-2 bis A-4 dokumentieren die ursprünglichen Zählungen unverändert.

Die frühere Ressourcenprüfung umfasste drei getrennte Prozesse je Methode und Betriebsart, insgesamt neun Sensor- und neun Replay-Prozesse mit je 300 s Aufnahme beziehungsweise Wiedergabe. Bewertet wurde [180,300) s. Ihre CPU-Werte sind zeitgewichtete Mittel aus ungefähr 100-ms-Proben; die Handyreihe verwendet dagegen Mediane der laufbezogenen Proben. Außerdem unterscheiden sich Vorverarbeitung, Recorder und Speicherhaltung. Die historischen Zahlen in Tabellen A-5 und A-6 bleiben deshalb gültig für ihren damaligen Messumfang, sind aber kein austauschbarer Schätzer des aktuellen Bedarfs.

7.8 Forschungsfragen und Nachweisstand

Tabelle 7-8: Bewertung der Forschungsfragen und Hypothesen

Gegenstand	Beobachtung	Bewertung
FF1: technische Auswahl	Raspberry Pi, ADXL345 und lokale Pipeline eingesetzt	Ausführbarkeit im untersuchten Umfang gezeigt; Messqualität nur teilweise abgesichert
FF2 / H1: Erkennungsqualität	Historisches F1 für AE nicht besser; Handyreihe ohne unabhängiges Zeitlabel	H1 nicht bestätigt; Handyreihe liefert keinen Ersatz-F1
Ergänzende Zeitfrage	AE-Ausgabe zuerst; IF im selben Fenster, RMS ein Fenster später	Beobachteter AE-Vorteil im gemeinsamen Lauf; keine allgemeine Rangfolge
FF3 / H2: Zeitbudget und Ressourcen	Historischer Fristtest ohne Überschreitungen; neue Verarbeitung deutlich unter Fensterbudget	Ausführbarkeit gestützt, keine Garantie harter Echtzeit oder Dauerbetrieb
FF4 / H3: geänderter Normalbetrieb	Frühere 50%-Versuche methoden-/aufnahmeabhängig	Keine einheitliche Bestätigung; Handyreihe enthält keinen neuen Drehzahlvergleich
Wiederholbarkeit	Wenige Anrufe und erhaltene Fehlanläufe	Pilotnachweis, keine statistisch belastbare Überlegenheit

Die ursprünglichen Anforderungen an Nachvollziehbarkeit, gleiche Eingaben, Datentrennung und Ressourcenmessung werden durch Dateien, Hashes und getrennte Vergleichsmodi gestützt. Die Anforderungen an vollständig verifizierte Messqualität, Drehzahl, physikalische Referenz und reproduzierbare Anregung bleiben teilweise offen. Eine nachträgliche Definition günstigerer Erfolgskriterien wird nicht vorgenommen.

Tabelle 7-9: Rückbindung an die unveränderten Anforderungen aus Kapitel 4

Anforderung	Stand der Handyreihe	Abgrenzung / Nachweis
A1 Messqualität	Teilnachweis	Integritätsflags geprüft; Bandbreite, Signal-Rausch-Abstand und Sensortakt nicht vollständig unabhängig abgesichert
A2 Datentrennung	Im dokumentierten Umfang erfüllt	Drei normale Trainingsdateien, eine getrennte normale Validierungsdatei, spätere Handyaufnahmen
A3 Einheitlicher Vergleich	Teilnachweis	Identische Eingaben und P99-Regel; unabhängige physikalische Zeitlabels fehlen
A4 Lokale Auswertung	Im dokumentierten Umfang erfüllt	Entscheidungen lokal auf dem Pi; kein Cloud-Dienst für die Inferenz
A5 Nachvollziehbarkeit	Im dokumentierten Umfang erfüllt	Laufkennungen, Rohdaten, Bündel-/Dateihashes und Softwareprovenienz vorhanden
A6 Modellkompatibilität	Im dokumentierten Umfang erfüllt	Keras-/TFLite-Prüfung und exakte Reproduktion der Live-Entscheidungen
A7 Latenzziel	Historisch geprüft; neuer Teilnachweis	Aktuelle Zeitverteilungen deutlich unter Fensterbudget; keine identische Wiederholung der früheren Fristmatrix
A8 Speicher und Rückstand	Teilnachweis	Kurze Läufe ohne protokollierten Abbruch/Rückstand; kein Dauerbetriebs- oder vollständig kontrollierter Lastnachweis
A9 Erkennungsqualität	In der Handyreihe offen	Keine belastbare Konfusionsmatrix ohne physische Referenz; historische Zustandskennzahlen bleiben separat
A10 Andere Normalbedingungen	Kein neuer Nachweis	Aktuell nur 75 % PWM; frühere 50%-Tests unter B6
A11 Reproduzierbarkeit	Teilnachweis	Protokoll und Daten erhalten; Handygeometrie und Wiederholungsgleichheit nicht vollständig bestätigt
A12 Live-GUI	Nicht Gegenstand dieser Messung	GUI vorhanden, Aufnahme ohne GUI; Zusatzaufwand nicht bestimmt
A13 Geordnetes Starten/Stoppen	Im dokumentierten Umfang erfüllt	Abgeschlossene Journale, gespeicherte Reste und keine verbliebenen vollständigen Fenster

8 Diskussion

8.1 Beobachteter Vorteil des Autoencoders

Für die konkrete Frage „Welche Methode gibt beim gemeinsamen Ereignis zuerst einen Alarm aus?“ lautet das Ergebnis dieses Laufs: Autoencoder vor Isolation Forest vor RMS. Der Autoencoder verbindet hier eine frühe Schwellenüberschreitung mit geringerem Verarbeitungsaufwand als der Isolation Forest. Gegenüber RMS liegt der entscheidende Unterschied auf der Erkennungsseite: RMS ist rechnerisch schneller, überschreitet aber erst beim nächsten Fenster seine Grenze. Der beobachtete Autoencoder-Vorteil ist dadurch sowohl im Zeitjournal als auch im Scoreverlauf nachvollziehbar.

Auf Fensterebene sind AE und IF im gemeinsamen Lauf gleichauf. Ein alleiniger „Erkennungssieg“ des AE gegenüber IF wäre daher ungenau, wenn damit ein früheres betroffenes Sensorfenster gemeint wäre. Der Vorsprung von 19,061 ms betrifft die verfügbare Ausgabe. Im zusätzlichen Offline-Vergleich ist AE auf einer von drei Rohaufnahmen ein Fenster früher als beide Baselines und auf den anderen beiden gleichauf. Das stützt seine Eignung für weitere Untersuchungen, reicht aber nicht für einen universellen Leistungsnachweis.

8.2 Schwellen, Alarmhäufigkeit und faire Bewertung

Eine niedriger aussehende MSE-Schwelle ist nicht automatisch falsch: Ihr Zahlenwert gehört zur standardisierten Eingabe und zur Rekonstruktionsfehlerverteilung des trainierten Modells. Ein hoher MSE im Betrieb bedeutet zunächst eine Abweichung von dieser Referenz. Mögliche Ursachen sind die beabsichtigte Anregung, veränderte Montage, Orientierung oder Normalbedingungen. Ein Schwellenwert darf nicht nur deshalb erhöht werden, weil ein aktueller Versuch viele Alarme liefert. Umgekehrt ist ein niedriger Schwellenwert kein unabhängiger Beleg hoher Qualität.

Die Normierung Score/Schwelle macht den Grenzübertritt sichtbar, gleicht aber die Verteilungen der drei Methoden nicht an. Ein Verhältnis von 3 beim Autoencoder ist nicht dreimal so viel physische Störung wie ein Verhältnis von 1 beim Isolation Forest. Die gemeinsame P99-Regel ist ein nachvollziehbarer Kalibrierungsansatz, garantiert aber bei kleinen und zeitlich abhängigen Normaldaten keine identische Sensitivität oder Spezifität im Test.

Mehr Alarmfenster können längere Empfindlichkeit, Nachschwingen oder Fehlalarme bedeuten. Ohne unabhängige Ereignisreferenz lässt sich dies nicht vollständig trennen. Ebenso ist „kein Alarm außerhalb der Bedienmarker“ ein nützlicher Kontrollbefund, aber kein statistischer Nachweis einer Fehlalarmrate von null. Alarmfenster desselben Anrufs dürfen nicht als unabhängige erfolgreiche Wiederholungen gezählt werden.

8.3 Vergleichbarkeit und interne Gültigkeit

Der gemeinsame Live-Lauf ist für die Rangfolge im konkreten Ereignis am aussagekräftigsten, weil alle Methoden dieselben Rohfenster erhalten. Er beantwortet jedoch nicht den isolierten Ressourcenbedarf jedes Verfahrens. Die Einzelprozesse helfen bei dieser zweiten Frage, sind aber aufgrund unterschiedlicher Eingaben, Laufdauern und fehlender Wiederholungsmatrix nur deskriptiv vergleichbar. Replay trennt die Wirkung der Eingabedaten von unterschiedlichen Anrufen, ersetzt aber keine physikalische Wiederholung.

Weitere Unsicherheit entsteht durch die Kopplung Handy-Tisch-Aufbau-Sensor. Position, Unterlage und Vibrationsmuster wurden nicht unabhängig quantifiziert. Das aktuelle Normalprofil ist setupbezogen; eine Änderung des mechanischen Wegs kann bereits die Normalverteilung verschieben. Die fehlende vollständige Bestätigung der finalen Handygeometrie und der Entwicklungsstatus der Messkette werden deshalb nicht durch einen positiven Einzelbefund aufgehoben.

Die nominelle Sensorrate, die Host-Zeitstempel und der physikalische Beginn sind verschiedene Größen. Die zeitliche Lage einer Störung innerhalb des etwa 0,62-s-Fensters beeinflusst, wann ein genügend auffälliges Fenster vollständig vorliegt. Eine sehr kurze Modellberechnung allein garantiert deshalb keine entsprechend kurze Ende-zu-Ende-Erkennung. Gleichzeitige Worker-Starts beseitigen außerdem weder Betriebssystem-Scheduling noch Konkurrenz in Python und nativen Bibliotheken.

8.4 Raspberry-Pi-Bedarf und Grenzen der Übertragung

Im beobachteten Einzelbetrieb benötigt RMS den geringsten Speicher und Rechenaufwand. Isolation Forest besitzt hier den größten Prozess-RSS und die längste Verarbeitung. Der Autoencoder ist deutlich sparsamer im beobachteten Prozessspeicher als IF und gibt im gemeinsamen Ereignis früher Alarm aus. Diese Kombination ist für die untersuchte Aufgabe günstig. Gegenüber RMS ist dagegen kein pauschaler Ressourcen- oder Energiegewinn nachgewiesen.

Die gemeinsame Ausführung bleibt in der kurzen Messung innerhalb des verfügbaren Fensterbudgets. Daraus folgt weder eine garantierte Reaktion unter jeder Systemlast noch ein Nachweis unbegrenzten Dauerbetriebs. Die Hardwaredaten selbst bleiben gleich; Last, Speicher und Temperaturen sind jedoch Betriebswerte und müssen für jeden Messmodus gesondert ausgewiesen werden. Die alten Messwerte werden deshalb nicht als unveränderliche Eigenschaften des Raspberry Pi dargestellt.

9 Ausblick

Für eine belastbare Fortsetzung ist zuerst die physische Referenz zu verbessern. Vibrationsbeginn und -ende sollten mit einer unabhängigen, zeitlich zugeordneten Referenz erfasst werden. Handyposition, Auflage und Anregungsmuster sind fest zu dokumentieren. Ein automatisiertes reproduzierbares Vibrationssignal wäre für Wiederholungen geeigneter als nur die manuelle Bestätigung eines Anrufs. Es bleibt dabei zu prüfen, welche reale Fehlerklasse durch eine äußere Tischanregung tatsächlich repräsentiert wird.

Danach sind mehrere unabhängige Ereignisse und längere Normalaufnahmen mit unverändert eingefrorenen Modellen nötig. Die Auswertung sollte vorab festlegen, wie ein Ereignis als erkannt gilt, wie Nachschwingen behandelt wird und welche Zeitabweichung akzeptiert wird. Eine Train-/Validierungs-/Testtrennung nach vollständigen Aufnahmen bleibt notwendig. Erst mit solchen Daten sind Erkennungsquote, Fehlalarmrate und Erkennungsverzögerung mit sinnvoller Unsicherheit vergleichbar.

Für den Raspberry-Pi-Vergleich sollten alle Methoden wiederholt auf denselben gespeicherten Daten in getrennten Prozessen laufen, ergänzt um gemeinsame Live-Versuche. Reihenfolge, Warmup, Messdauer, CPU-Konfiguration und Hintergrundlast sind festzulegen beziehungsweise zu rotieren. GUI-Last und Modellstart sind bei Bedarf getrennt zu messen. Elektrische Leistung erfordert eine zusätzliche Spannungs-/Strommessung mit klarer Systemgrenze; erst dann sind Energie pro Fenster oder Energie pro erkanntem Ereignis sinnvoll bestimmbar.

Der Autoencoder ist aufgrund der hier beobachteten frühen Ausgabe ein begründeter Kandidat für diese Weiterführung. Eine Auswahl für einen produktiven Einsatz muss jedoch gemeinsam Erkennungsqualität, Fehlalarme, Zeitverhalten, Rechenbedarf und Messkettenqualität berücksichtigen. Die nächsten Versuche sollen die vorliegende Interpretation prüfen und nicht nur einen bevorzugten Sieger bestätigen.

10 Fazit

Die aktualisierte Arbeit zeigt einen nachvollziehbaren Vergleich von RMS, Isolation Forest und TFLite-Autoencoder bei einer über den Tisch eingeleiteten Handyvibration. Raspberry Pi 5, ADXL345 und Lüfterhardware bleiben unverändert; der aktuelle Betriebspunkt ist 75 % PWM. Die neue Anregung benötigt keine Platte. Normalprofil, Vorverarbeitung und Versuchsdaten sind vom früheren v4-Stand getrennt dokumentiert.

Im gemeinsamen Live-Lauf liegen für alle drei Verfahren dieselben 197 Rohfenster vor. Der Autoencoder liefert den ersten Alarm. Isolation Forest markiert dasselbe Fenster 108, gibt seine Entscheidung aber 19,061 ms später aus. RMS markiert erst Fenster 109 und folgt 615,659 ms nach der ersten AE-Ausgabe. Der Autoencoder erzielt damit für das konkret gemessene Kriterium der frühesten Alarmausgabe das beste Ergebnis dieses Laufs. Der Replay-Vergleich ergänzt einen früheren AE-Fensterbeginn auf einer von drei Aufnahmen bei zwei Gleichständen.

Der Ressourcenvergleich zeigt einen anderen Schwerpunkt: RMS wird am schnellsten berechnet und benötigt den kleinsten beobachteten Prozessspeicher. Isolation Forest ist hier aufwendiger als der Autoencoder. Der gemeinsame Prozess erreicht einen CPU-Median von 13,04 % eines logischen Kerns und maximal 174,42 MiB abgetasteten RSS, ohne protokollierte Fensterverluste. Das belegt die lokale Ausführbarkeit im beobachteten Umfang, nicht den elektrischen Energieverbrauch oder einen unbegrenzten Dauerbetrieb.

Die ursprüngliche Hypothese eines höheren Autoencoder-F1 wird durch den neuen Zeitbefund nicht bestätigt: Für die Handyreihe fehlen unabhängig gemessene physische Ereignisgrenzen, für die frühere Zustandsfolge bleiben die negativen Ergebnisse bestehen. Der Bericht schließt daher mit einem konkret belegten Autoencoder-Vorteil bei der beobachteten Alarmausgabe, einer transparenten Ressourcenabwägung und offenen Nachweisen für allgemeine Zuverlässigkeit, Wiederholbarkeit und absolute Fehler-Erkennungszeit.

Anhang

Digitale Belege und Fassungszuordnung

Die folgenden Pfade sind relativ zum Projektstamm masterarbeit-edge-ai zu lesen. B1 bis B11 dokumentieren frühere Entwicklungs-, Prüf- und Berichtsstände. B12 ist die Quelle der Handyreihe; B13 dokumentiert diese Word-Revision. Historische Werte bleiben ihren ursprünglichen Eingaben, Zeitabschnitten und Modellständen zugeordnet.

Tabelle A-1: Nachweisverzeichnis

ID	Inhalt	Projektpfad
B1	Eingefrorenes Modellpaket und Trainingsnachweis	results/upright_v4_training_pilot_20260911_130614/run_001/
B2	Drei unabhängige normale 75%-Tests	results/upright_v4_independent_normal_20260911_133047/
B3	Zusätzlicher Normaltest und dokumentierter abgebrochener Plattenstart	results/upright_v4_frozen_airflow_test_20260911_164040/
B4	Vollständiger unabhängiger Test Normal → Platte → Normal	results/upright_v4_frozen_airflow_test_20260912_073015/
B5	Explorative Fehleranalyse auf identischen Modellfenstern	results/upright_v4_exploratory_diagnosis_20260912_081728/
B6	Drei unabhängige normale 50%-Tests	results/upright_v4_second_pwm50_20260912_091000/
B7	Abschließende Sensor-/Replay-Laufzeitprüfung und Erhaltungsnachweis	results/upright_v4_finalization_20260912_103145/
B8	Vor dem Training erstellte Spektralprüfung der Entwicklungsaufnahmen	results/upright_v4_spectral_review_20260911_114502/
B9	Fachreview und unveränderte Zählung je Aufnahme vom 12.09.2026	results/upright_v4_followup_closure_20260912/
B10	Abschluss als begrenzte Pilotstudie: Quellen-, Statistik- und Konsistenzprüfung; Änderungs- und Erhaltungsnachweis	results/upright_v4_pilot_manuscript_20260913_064035/
B11	Gezielte GUI-Dokumentation: Quell-/Profilprüfung, historische Nachweise, hardwarefreie Tests und Word-Änderungsnachweis	results/gui_documentation_review_20260913_085052/
B12	Handyversuche, eingefrorenes Bündel, Einzel-/Replay-/Live-Vergleich und Präsentation	results/phone_vibration_pilot_20260919/
B13	Word-Revision, neue Diagramme, Quellenhashes und Layoutprüfung	results/phone_vibration_word_revision_20260919/

Das historische Modellpaket pilot_bundle.json im Unterverzeichnis frozen von B1 besitzt den SHA-256-Wert cec1859bfb354bafef77a619e6d2c1ffd85b50787c87595aba9a1e20a80cdae5. Das Bündel der Handyreihe ist über den in den Laufdateien und im Replay gespeicherten

SHA-256-Wert

dbfad8529b642d1e4c74119b3865b0600c3a7668198328e1112707c7502b2eb1 zugeordnet. Vollständige Artefaktmanifeste stehen in den jeweiligen Verzeichnissen. Diese Kennungen bezeichnen unterschiedliche Modell-/Messkettenstände und sind nicht austauschbar.

Die Ausgangs-Worddatei ist unter docs/backups/Akz_Masterarbeit_Bericht(3)_vor_handyversuch_20260919.docx erhalten. B13 enthält die neue Textgrundlage, Diagramme als PNG/PDF, abgeleitete Kennwerte, Quellenhashes und den Änderungs-/Layoutnachweis. Rohdaten, Modellgewichte und Schwellen werden durch die Dokumentrevision nicht verändert.

Erhaltene historische Erkennungsergebnisse

Die folgenden Tabellen übernehmen die früheren Zählungen unverändert. Positiv bezeichnet den vorab festgelegten Zustand airflow_modified der Plattenfolge vom 12. September; negativ sind die beiden Normalreferenzen derselben Folge. Bewertet werden jeweils 194 gültige Fenster im Abschnitt [180,300) s einer 300-s-Aufnahme. Diese Definition gilt nur für die historische Reihe und wird nicht auf die Handy-Bedienmarker übertragen.

Tabelle A-2: Historische Verwechslungszahlen der einen Plattenfolge auf identischen Fenstern

Methode	TP: Änderung erkannt	FN: Änderung übersehen	FP: normal auffällig	TN: normal unauffällig
RMS	7	187	1	387
Isolation Forest	4	190	0	388
Autoencoder	0	194	55	333

Tabelle A-3: Historische deskriptive Kennzahlen für das Zustandslabel; Prozent

Methode	Precision	Recall	F1	Fehlalarmrate	Accuracy
RMS	87,50	3,61	6,93	0,26	67,70
Isolation Forest	100,00	2,06	4,04	0,00	67,35
Autoencoder	0,00	0,00	0,00	14,18	57,22

Je Methode liegen 582 gültige Fenster vor, davon 388 aus zwei Normalaufnahmen und 194 aus einer veränderten Aufnahme. Die Precision von 100 % beim IF betrifft nur vier Alarme; 190 veränderte Fenster bleiben unmarkiert. Ein immer normales Verfahren erreichte auf dieser Klassenverteilung bereits 66,67 % Accuracy. Die Kennzahlen sind keine allgemeine Defekterkennungsleistung. Eine positive Aufnahme und zeitlich abhängige Fenster erlauben keinen unabhängigen Wiederholbarkeitsnachweis.

Tabelle A-4: Historische Zählung je Aufnahme und Methode im Abschnitt [180,300) s

Aufnahme / Methode	gültig / ungültig	TP / FN	FP / TN	Anteil [%]	Bedeutung
Normal vorher / RMS	194 / 0	0 / 0	1 / 193	0,52	Fehlalarme
Normal vorher / IF	194 / 0	0 / 0	0 / 194	0,00	Fehlalarme
Normal vorher / AE	194 / 0	0 / 0	51 / 143	26,29	Fehlalarme
Luftstrom verändert / RMS	194 / 0	7 / 187	0 / 0	3,61	Änderung markiert
Luftstrom verändert / IF	194 / 0	4 / 190	0 / 0	2,06	Änderung markiert
Luftstrom verändert / AE	194 / 0	0 / 194	0 / 0	0,00	Änderung markiert
Normal nachher / RMS	194 / 0	0 / 0	0 / 194	0,00	Fehlalarme
Normal nachher / IF	194 / 0	0 / 0	0 / 194	0,00	Fehlalarme
Normal nachher / AE	194 / 0	0 / 0	4 / 190	2,06	Fehlalarme

Die Berechnung und Quellenhashes der aufnahmespezifischen Werte stehen unter B9 in per_record_metrics.json; die Zustandsauswertung steht unter B7 in controlled_state_descriptive_metrics.json. Die früheren Normaltests bei 50 % PWM bleiben unter B6 erhalten. Sie zeigen keinen einheitlichen Anstieg der Fehlalarmrate aller Verfahren; der Autoencoder steigt in der damaligen zusammengefassten Auswertung von 5,50 % auf 6,01 %, während die beiden Baselines sinken. Dieser Befund gehört nicht zur neuen Handyreihe.

Erhaltene historische Raspberry-Pi-Messwerte

Die Tabellen A-5 und A-6 bleiben zahlenmäßig unverändert gegenüber der Ausgangsfassung. Ihre Messmatrix umfasst drei neue Prozesse je Methode und Betriebsart, 300 s Erfassung beziehungsweise zeitgetreues Replay und den aktiven Bewertungsabschnitt [180,300) s. Zwanzig synthetische Aufwärmaufrufe liegen vor dem Messbeginn. Reihenfolgen wurden rotiert. Sensorstarts verwenden 75 % PWM bei 25 kHz; nach den damaligen Sensorläufen wurde 0 % zurückgelesen. Diese historische Stopplilogik darf nicht als Verhalten der Handyrecorder beschrieben werden.

Tabelle A-5: Historische Latenzen; Bereich von drei getrennten Prozessen je Zeile

Betriebsart / Methode	P99 gesamt [ms]	P99 Rechenkern [ms]	größtes Maximum gesamt [ms]	Überschreitungen
Sensor / RMS	6,113–6,203	0,045–0,049	7,908	0
Sensor / Isolation Forest	27,896–28,577	22,004–22,049	29,234	0
Sensor / TFLite-Autoencoder	6,261–7,062	0,086–0,114	8,207	0
Replay / RMS	6,077–6,558	0,045–0,057	7,519	0
Replay / Isolation Forest	27,802–29,325	21,884–22,957	32,462	0
Replay / TFLite-Autoencoder	6,236–6,587	0,099–0,116	6,785	0

Die damalige gesamte Latenz beginnt beim Host-Leseabschluss des letzten XYZ-Punkts und endet vor dem Schreiben der Entscheidung ins Journal. Sie umfasst unter anderem Übergabe, Fensteraufbau, Qualitätsprüfung, fensterweise Mittelwertentfernung, Queue,

Standardisierung und Auswertung. Die Rechenkernzeit bezeichnet nur die jeweils isoliert instrumentierte Berechnung, beim Autoencoder Interpreter.invoke. Sie ist nicht identisch mit processing_ms des aktuellen Handyrecorders. Die historische H2-Auswertung dokumentiert keine Fristüberschreitungen im untersuchten Umfang; sie ist kein Beleg harter Echtzeit.

Tabelle A-6: Historische Ressourcen im aktiven Abschnitt [180,300) s; Bereich von drei Prozessen

Betriebsart / Methode	mittlere CPU [% eines Kerns]	maximales RSS [MiB]	gültig / ungültig gesamt	verworfenen Fenster
Sensor / RMS	5,02–5,12	75,47–75,48	582 / 0	0
Sensor / Isolation Forest	8,59–8,62	171,00–171,00	582 / 0	0
Sensor / TFLite-Autoencoder	5,02–5,16	86,30–86,42	582 / 0	0
Replay / RMS	1,65–2,04	142,12–143,61	582 / 0	0
Replay / Isolation Forest	5,06–5,32	238,47–238,48	582 / 0	0
Replay / TFLite-Autoencoder	1,80–1,99	153,38–153,41	582 / 0	0

Die historische CPU-Auslastung ist zeitgewichtet aus ungefähr 100-ms-Monitorproben berechnet. 100 % entsprechen einem logischen Kern. Der erste CPU-Wert ohne vorherigen Monitorzeitstempel ist aus dieser Gewichtung ausgeschlossen. RSS erfasst den vollständigen Python-Prozess. Beim Replay waren die Quelldaten bereits als DataFrame und Datensatzliste geladen; dieser Zusatzspeicher ist kein reiner Modellbedarf. Modellstart und abschließendes erneutes Laden von Daten liegen außerhalb des dargestellten aktiven Ressourcenverlaufs. Die Zahlen dürfen nicht unmittelbar gegen die anders definierten CPU-Mediane und abgetasteten RSS-Werte der Handyreihe als Optimierungsnachweis verrechnet werden.

Umgebung der dokumentierten Ausführung: Raspberry Pi 5 mit vier logischen Kernen, 64-Bit-Linux und Python 3.13.5. Die Laufdateien der Handyaufnahme nennen NumPy 2.5.1, scikit-learn 1.9.0, ai-edge-litert 2.1.6 und psutil 7.2.2. Der Trainingsprofil-Snapshot stammt aus einer separat erfassten Umgebung und nennt unter anderem NumPy 2.5.2 und TensorFlow 2.21.0. Die jeweilige Provenienzdatei hat Vorrang vor einer pauschalen Gleichsetzung aller Softwarestände. Die Hardwareauswahl ist davon unberührt.

Lesen der aktuellen Messdateien

raw.csv enthält sample_index, window_index, monotone und UTC-Hostzeit, x_g, y_g, z_g und Qualitätsmerkmale. Ein vollständiges Modellfenster besteht aus 128 aufeinanderfolgenden XYZ-Punkten desselben Laufs. Anders als in der früheren 300-s-Matrix wird im Handyversuch nicht nur [180,300) s ausgewertet. Die ganze nach Warmup gestartete Aufnahme liefert vollständige Fenster; Restpunkte bleiben dokumentiert.

decisions.csv ordnet Score, eingefrorene Schwelle, Entscheidung und Fehlerstatus dem Fenster zu. Im gemeinsamen Lauf kommen method, input_sha256, dispatch_ns, worker_start_ns und decision_ns hinzu. window_to_decision_ms verwendet den Host-Abschluss des letzten Fensterpunkts. resources.csv gehört zum gemeinsamen Prozess, nicht

zu einer isolierten Methode. CPU-Prozente, RSS, Temperatur und Takt sind Messproben des beobachteten Prozesses und Systems.

cues.jsonl speichert die Bedienereignisse; run.json beschreibt Laufstatus, Sensor, PWM und Erfassungsintegrität. Eine Null oder ein fehlender Wert für true_detection_delay_ms, detection_rate beziehungsweise false_alarm_rate bedeutet nicht null Millisekunden oder null Prozent, sondern einen nicht bestimmbaren Nachweis. UTC-Einträge sind keine nachträglich erfundenen physikalischen Ereignisgrenzen. Bilder und Präsentationsansichten sind Ergebnisdarstellungen, keine zusätzlichen Messungen.

KI-Unterstützung und fachliche Verantwortung

Bei der Bearbeitung wurde Codex von OpenAI als KI-gestütztes Werkzeug für Softwareentwicklung und Fehlersuche, Datenauswertung und Grafikaufbereitung, Quellenprüfung sowie die Formulierung, Überarbeitung und Formatprüfung des Berichts verwendet. Dies umfasst auch generierte und überarbeitete Textpassagen; die Unterstützung beschränkt sich nicht auf Rechtschreibkorrekturen. Die physische Einrichtung und die im Kontrolljournal ausgewiesenen Sichtbeobachtungen stammen aus Nutzerangaben. KI-Ausgaben gelten nicht als eigenständige wissenschaftliche Quellen oder als Ersatz für Messnachweise.

Der digitale Nachweis B13 dokumentiert die aktuelle Überarbeitung anhand der archivierten Versuchsdaten; B10 und B11 enthalten frühere Unterstützungs- und Änderungsnachweise. Eine positive Darstellung des beobachteten Autoencoder-Vorteils darf weder fehlende Messungen ersetzen noch historische negative Ergebnisse verändern. Die persönliche fachliche Prüfung, die Eigenständigkeitserklärung und die Einhaltung der geltenden Hilfsmittelregeln können nicht durch die Software bestätigt werden.

Literaturverzeichnis

- Analog Devices (2022). ADXL345: 3-Axis, ± 2 g/ ± 4 g/ ± 8 g/ ± 16 g Digital Accelerometer. Datenblatt, Revision G, Mai 2022. <https://www.analog.com/media/en/technical-documentation/data-sheets/adxl345.pdf> (Abruf: 12.09.2026.)
- Analog Devices (2025). ADXL354/ADXL355: Low Noise, Low Drift, Low Power, 3-Axis MEMS Accelerometers. Datenblatt, Revision D, Juni 2025. https://www.analog.com/media/en/technical-documentation/data-sheets/adxl354_adxl355.pdf (Abruf: 12.09.2026.)
- Antonini, M., Pincheira, M., Vecchio, M., & Antonelli, F. (2023). An Adaptable and Unsupervised TinyML Anomaly Detection System for Extreme Industrial Environments. *Sensors*, 23(4), 2344. <https://doi.org/10.3390/s23042344> (Abruf: 12.09.2026.)
- ARCTIC (o. J.). Specifications: P12 Pro PST. Technisches Datenblatt. https://www.arctic.de/media/2c/de/c6/1750758983/Spec_Sheet_P12_Pro_PST_EN.pdf (Abruf: 12.09.2026.)
- Buttazzo, G. C. (2011). *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications* (3. Aufl.). Springer. <https://doi.org/10.1007/978-1-4614-0676-1> (Abruf: 12.09.2026.)
- Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly Detection: A Survey. *ACM Computing Surveys*, 41(3), Artikel 15, 1–58. <https://doi.org/10.1145/1541880.1541882> (Abruf: 12.09.2026.)
- Department for Communities and Local Government (2009). *Multi-criteria analysis: a manual*. London. ISBN 978-1-4098-1023-0. <https://www.gov.uk/government/publications/multi-criteria-analysis-manual-for-making-government-policy> (Abruf: 12.09.2026.)
- Garay, C. E., Miranda Bonomi, F. A., Mansilla, G. N., Fagre, M., Guzmán, S. G., Ritorto, P. A., Perez, F. I., & Katz, M. (2026). A Multimodal TinyML-Based Predictive Maintenance Architecture for Industrial IoT in the 6G Era. *Sensors*, 26(14), 4536. <https://doi.org/10.3390/s26144536> (Abruf: 12.09.2026.)
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. Insbesondere Kapitel 14: Autoencoders. <https://www.deeplearningbook.org/> (Abruf: 12.09.2026.)
- Google (2026a). Convert TensorFlow models. LiteRT-Dokumentation, Stand 28.05.2026. https://developers.google.com/edge/litert/conversion/tensorflow/convert_tf (Abruf: 12.09.2026.)
- Google (2026b). Post-training quantization. LiteRT-Dokumentation, konsultierte Fassung 2026. https://developers.google.com/edge/litert/conversion/tensorflow/quantization/post_training_quantization (Abruf: 12.09.2026.)
- Google (2026c). Quickstart for Linux-based devices with Python. LiteRT-Dokumentation, konsultierte Fassung 2026. <https://developers.google.com/edge/litert/microcontrollers/python>
<https://developers.google.com/edge/api/tflite/python/tf/lite/Interpreter> (Abruf: 12.09.2026.)
- Google (2026d). Install TensorFlow with pip. TensorFlow-Dokumentation, konsultierte Fassung 2026. <https://www.tensorflow.org/install/pip> (Abruf: 12.09.2026.)
- Jardine, A. K. S., Lin, D., & Banjevic, D. (2006). A review on machinery diagnostics and prognostics implementing condition-based maintenance. *Mechanical Systems and Signal Processing*, 20(7), 1483–1510. <https://doi.org/10.1016/j.ymssp.2005.09.012> (Abruf: 12.09.2026.)
- Kaufman, S., Rosset, S., Perlich, C., & Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4), Artikel 15. <https://doi.org/10.1145/2382577.2382579> (Abruf: 12.09.2026.)
- Knap, P., Jachymczyk, U., & Lalik, K. (2026). Leakage-Safe, Reproducible Benchmarking for Vibration-Based Fault Diagnosis. *PHM Society European Conference*, 9(1), 1–8. <https://doi.org/10.36001/phme.2026.v9i1.4924> (Abruf: 12.09.2026.)

- Lessmeier, C., Kimotho, J. K., Zimmer, D., & Sextro, W. (2016). Condition Monitoring of Bearing Damage in Electromechanical Drive Systems by Using Motor Current Signals of Electric Motors: A Benchmark Data Set for Data-Driven Classification. PHM Society European Conference, 3(1).
<https://doi.org/10.36001/phme.2016.v3i1.1577>.
https://mb.uni-paderborn.de/fileadmin-mb/kat/PDF/Veroeffentlichungen/20160703_PHME16_CM_bearing.pdf (Abruf: 12.09.2026.)
- Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation Forest. Proceedings of the Eighth IEEE International Conference on Data Mining, 413–422. <https://doi.org/10.1109/ICDM.2008.17> (Abruf: 12.09.2026.)
- NASA (2023). 6.8 Decision Analysis. Webfassung des NASA Systems Engineering Handbook. <https://www.nasa.gov/reference/6-8-decision-analysis/> (Abruf: 12.09.2026.)
- NVIDIA (o. J.). Jetson Orin Nano Super Developer Kit. Technische Spezifikationen. <https://www.nvidia.com/de-de/autonomous-machines/embedded-systems/jetson-orin/nano-super-developer-kit/> (Abruf: 12.09.2026.)
- ONNX Runtime developers (o. J.). Python. ONNX Runtime-Dokumentation zur Installation und Ausführung. <https://onnxruntime.ai/docs/get-started/with-python.html> (Abruf: 12.09.2026.)
- Oriental Motor (o. J.). Servo Motor Features Overview. <https://www.orientalmotor.com/servo-motors/technology/servo-motor-features.html> (Abruf: 12.09.2026.)
- PCB Piezotronics (o. J.). Model 356A32: Triaxial Accelerometer. Technische Spezifikationen. <https://www.pcb.com/products?m=356a32> (Abruf: 12.09.2026.)
- Pittino, F., Puggl, M., Moldaschl, T., & Hirschl, C. (2020). Automatic Anomaly Detection on In-Production Manufacturing Machines Using Statistical Learning Methods. Sensors, 20(8), 2344. <https://doi.org/10.3390/s20082344> (Abruf: 12.09.2026.)
- PyTorch (o. J.). ExecuTorch Documentation. Dokumentation zum Modellexport und zur lokalen Ausführung. <https://docs.pytorch.org/executorch/stable/index.html> (Abruf: 12.09.2026.)
- Randall, R. B., & Antoni, J. (2011). Rolling element bearing diagnostics—A tutorial. Mechanical Systems and Signal Processing, 25(2), 485–520. <https://doi.org/10.1016/j.ymssp.2010.07.017> (Abruf: 12.09.2026.)
- Raspberry Pi Ltd. (o. J.). Raspberry Pi 5. Produktbeschreibung und technische Spezifikationen. <https://www.raspberrypi.com/products/raspberry-pi-5/>
<https://datasheets.raspberrypi.com/rpi5/raspberry-pi-5-product-brief.pdf> (Abruf: 12.09.2026.)
- Raspberry Pi Ltd. (2026). Raspberry Pi 4 Model B. Product Brief, April 2026. <https://pip-assets.raspberrypi.com/categories/545-raspberry-pi-4-model-b/documents/RP-008344-DS-1-raspberry-pi-4-product-brief> (Abruf: 12.09.2026.)
- Ren, H., Anicic, D., & Runkler, T. A. (2021). TinyOL: TinyML with Online-Learning on Microcontrollers. International Joint Conference on Neural Networks, 1–8. <https://doi.org/10.1109/IJCNN52387.2021.9533927> (Abruf: 12.09.2026.)
- Ruff, L., Kauffmann, J. R., Vandermeulen, R. A., Montavon, G., Samek, W., Kloft, M., Dietterich, T. G., & Müller, K.-R. (2021). A Unifying Review of Deep and Shallow Anomaly Detection. Proceedings of the IEEE, 109(5), 756–795. <https://doi.org/10.1109/JPROC.2021.3052449> (Abruf: 12.09.2026.)
- scikit-learn developers (o. J. a). Common pitfalls and recommended practices. Benutzerhandbuch. https://scikit-learn.org/stable/common_pitfalls.html (Abruf: 12.09.2026.)
- scikit-learn developers (o. J. b). Cross-validation: evaluating estimator performance. Insbesondere Aufteilung gruppierter Daten. https://scikit-learn.org/stable/modules/cross_validation.html (Abruf: 12.09.2026.)

- scikit-learn developers (o. J. c). Metrics and scoring: quantifying the quality of predictions. https://scikit-learn.org/stable/modules/model_evaluation.html (Abruf: 12.09.2026.)
- scikit-learn developers (o. J. d). IsolationForest. API-Dokumentation. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html> (Abruf: 12.09.2026.)
- SciPy developers (o. J. a). Discrete Fourier Transforms (scipy.fft). SciPy-Benutzerhandbuch. <https://docs.scipy.org/doc/scipy/tutorial/fft.html> (Abruf: 12.09.2026.)
- SciPy developers (o. J. b). Signal Processing (scipy.signal), Abschnitt Spectral Analysis. SciPy-Benutzerhandbuch. <https://docs.scipy.org/doc/scipy/tutorial/signal.html#spectral-analysis> (Abruf: 12.09.2026.)
- Shi, W., Cao, J., Zhang, Q., Li, Y., & Xu, L. (2016). Edge Computing: Vision and Challenges. IEEE Internet of Things Journal, 3(5), 637–646. <https://doi.org/10.1109/JIOT.2016.2579198> (Abruf: 12.09.2026.)
- STMicroelectronics (2026). Migrating applications from STM32F7 to STM32H743/H753 MCUs. Application Note AN4936, Fassung vom 09.01.2026. https://www.st.com/resource/en/application_note/an4936-migrating-applications-from-stm32f7-to-stm32h743h753-mcus-stmicroelectronics.pdf <https://www.st.com/en/microcontrollers-microprocessors/stm32h743-753.html> (Abruf: 12.09.2026.)
- Universität Paderborn, Lehrstuhl Konstruktions- und Antriebstechnik (o. J.). Bearing Data Center. Beschreibung des Lagerprüfstands und des Datensatzes. <https://mb.uni-paderborn.de/en/kat/research/bearing-datacenter> (Abruf: 12.09.2026.)